CSE 3230 · LAB 4 HOME TASK · বাংলা স্টাডি গাইড

University Management System

Spring Boot REST API · Spring Security · Docker Compose · Swagger UI — প্রতিটি ফাইলের লাইন-বাই-লাইন ব্যাখ্যা

ভাষা	Java 21	**Framework**	Spring Boot 3.5.16 (Spring Security 6.5, Hibernate 6.6)
Database	MySQL 8.4 (Docker), H2 (শুধু টেস্ট)	**যাচাই**	১৬টি টেস্ট পাস · demo.sh ও Swagger UI আসল Docker stack-এ চালানো

কীভাবে পড়বেন: অংশ ১ (অধ্যায় ১–৮) পুরো সিস্টেমের ধারণা দেয়। অংশ ২ (অধ্যায় ৯–২১) প্রতিটি ফাইল ছোট ছোট অংশে ভাগ করে দেখায় — প্রতিটি code অংশের ঠিক নিচে নীল বাক্সে লেখা আছে সেই লাইনগুলো কী করে এবং কেন। লাইন নম্বরগুলো আসল ফাইলের লাইন নম্বর, তাই editor-এ ফাইল খুলে মিলিয়ে দেখাতে পারবেন। অংশ ৩-এ চালানোর নিয়ম, ভাইভা প্রশ্নোত্তর ও সীমাবদ্ধতা।

সূচিপত্র

১. অ্যাসাইনমেন্ট কী চেয়েছে এবং কোথায় করা হয়েছে
২. ব্যবহৃত প্রযুক্তি ও কেন
৩. ফোল্ডার কাঠামো ও লেয়ার
৪. Spring-এর মূল ধারণাগুলো (সব ফাইলেই লাগবে)
৫. একটি request-এর পুরো যাত্রা
৬. ডাটাবেস ডিজাইন
৭. Spring Security-র ধারণা
৮. Docker-এর ধারণা
৯. pom.xml — Maven build ফাইল
১০. UmsApplication ও model/ — entity (ডাটাবেস টেবিল)
১১. repository/ — Spring Data দিয়ে ডাটাবেস অ্যাক্সেস
১২. dto/ — JSON-এর আকার
১৩. exception/ — একই ধরনের JSON error
১৪. service/ — ব্যবসায়িক নিয়ম ও transaction
১৫. controller/ — REST endpoint
১৬. security/ — Spring Security
১৭. config/ — প্রথম admin ও Swagger
১৮. application.yml — কনফিগারেশন
১৯. Dockerfile ও compose.yaml
২০. স্বয়ংক্রিয় টেস্ট — UniversityApiTests.java
২১. demo.sh — curl দিয়ে লাইভ ডেমো
২২. চালানো ও ডেমো দেখানো
২৩. সম্ভাব্য ভাইভা প্রশ্ন ও উত্তর
২৪. সীমাবদ্ধতা ও উন্নতির সুযোগ

১. অ্যাসাইনমেন্ট কী চেয়েছে এবং কোথায় করা হয়েছে

চাহিদা	কোথায় করা হয়েছে	প্রমাণ
Admin, Teacher, Student-এর CRUD REST API	`AdminController`, `TeacherController`, `StudentController` — প্রত্যেকটিতে GET (সব), GET (একটি), POST, PUT, DELETE	টেস্ট `fullCrudLifecycleForStudent`
সঠিক REST নিয়ম, JSON, HTTP status code	URL-এ বহুবচন noun (`/api/teachers/5`), method দিয়ে কাজ বোঝানো, create-এ 201 + `Location`, delete-এ 204, error-এ JSON (`ApiError`)	অধ্যায় ১৩, ১৫
Login / Logout	Login: `POST /api/auth/login` (`AuthController`)। Logout: `POST /api/auth/logout` (`SecurityConfig`-এ Spring-এর LogoutFilter)	টেস্ট `loginWith…`, `logoutInvalidatesSession`
Password hashing	`BCryptPasswordEncoder` (`SecurityConfig`), সেভের আগে hash (`UserAccountService`)	MySQL-এ `$2a$10$…`
Role-based access control	`SecurityConfig`-এর `authorizeHttpRequests` নিয়মগুলো	টেস্ট `adminCan…`, `teacherCan…`, `studentCan…`
Admin সব, Teacher শুধু Teacher, Student শুধু Student	`/api/admins/**` → ADMIN; `/api/teachers/**` → ADMIN, TEACHER; `/api/students/**` → ADMIN, STUDENT	demo.sh ধাপ ৫, ৬
Dockerfile ও compose.yaml	দুই-ধাপের (multi-stage) Dockerfile; compose-এ `db` (MySQL) ও `app`	`docker compose up --build`

বাড়তি যা করা হয়েছে: request validation (400), username/email/studentId ডুপ্লিকেট আটকানো (409), চালু হওয়ার সময় প্রথম admin ও ক্রমিক id-সহ ৭ জন demo user (admin, teacher, student) স্বয়ংক্রিয়ভাবে তৈরি, শেষ admin মুছতে না দেওয়া, response-এ কখনো password না পাঠানো, Swagger UI (ব্রাউজারে টেস্ট), ১৬টি স্বয়ংক্রিয় টেস্ট।

২. ব্যবহৃত প্রযুক্তি ও কেন

প্রযুক্তি	এখানে কী কাজ করে	কেন এটা
Java 21	প্রোগ্রামিং ভাষা	সর্বশেষ LTS (দীর্ঘমেয়াদি সাপোর্ট); `record`, text block আছে
Spring Boot 3.5.16	ভেতরে Tomcat সার্ভার চালায়, সব object জোড়া লাগায়, config পড়ে	পরিপক্ব 3.x লাইনের সর্বশেষ সংস্করণ; সবচেয়ে বেশি টিউটোরিয়াল
Spring Web (MVC)	`@RestController`, URL mapping, Jackson দিয়ে JSON	Spring-এ REST API বানানোর আদর্শ উপায়
Spring Data JPA + Hibernate	Java class ↔ টেবিল, SQL নিজে থেকে তৈরি	প্রায় কোনো SQL লিখতে হয় না
MySQL 8.4	আসল ডাটাবেস (Docker-এ)	MySQL-এর LTS লাইন
Spring Security 6.5	Authentication, authorization, BCrypt	Spring-এর স্ট্যান্ডার্ড security framework
Bean Validation	`@NotBlank`, `@Email` ইত্যাদি	if-else ছাড়াই ইনপুট যাচাই
springdoc (Swagger UI)	ব্রাউজারে প্রতিটি endpoint চালানোর পেজ	Postman ছাড়াই ডেমো
H2, JUnit 5, MockMvc	মেমরিতে ডাটাবেস দিয়ে স্বয়ংক্রিয় টেস্ট	MySQL ছাড়াই কয়েক সেকেন্ডে টেস্ট
Docker, Compose, Maven wrapper	প্যাকেজ ও চালানো; `./mvnw` নিজেই Maven নামায়	সব মেশিনে একই পরিবেশ

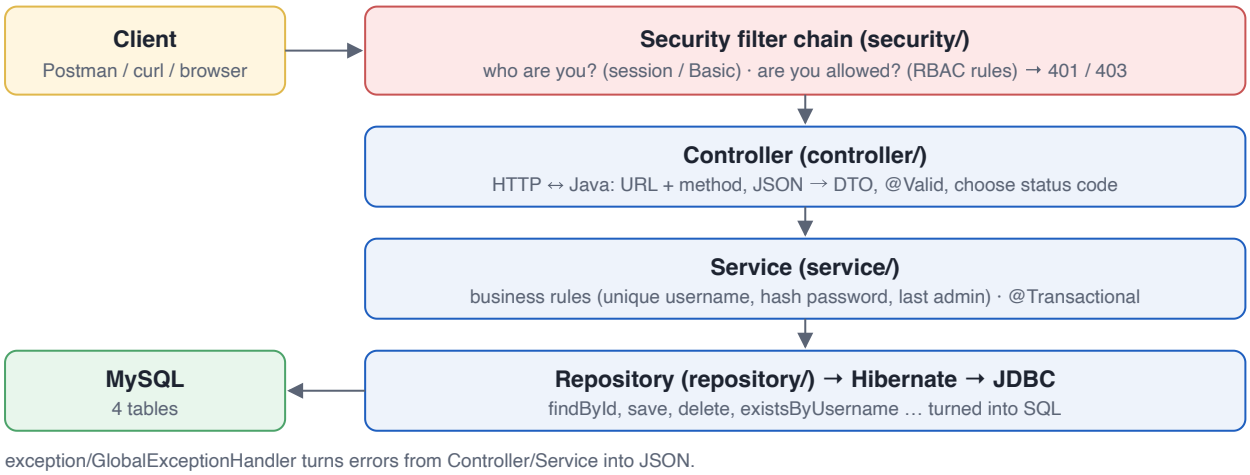
৩. ফোল্ডার কাঠামো ও লেয়ার

```
university-management-system/
├── pom.xml              Maven: dependency, Java version
├── mvnw, .mvn/          Maven wrapper
├── Dockerfile           app-এর image বানানোর নিয়ম
├── compose.yaml         app + MySQL একসাথে
├── .env.example         compose-এর পাসওয়ার্ড টেমপ্লেট
├── demo.sh              curl দিয়ে পুরো ডেমো
├── docs/                এই গাইড
└── src/
    ├── main/java/com/university/ums/
    │   ├── UmsApplication.java  main()
    │   ├── model/       টেবিল (entity)
    │   ├── repository/  ডাটাবেস অ্যাক্সেস
    │   ├── dto/         JSON-এর আকার
    │   ├── service/     ব্যবসায়িক নিয়ম
    │   ├── controller/  HTTP endpoint
    │   ├── security/    Spring Security
    │   ├── exception/   error → JSON
    │   └── config/      প্রথম admin, Swagger
    ├── main/resources/application.yml
    └── test/            টেস্ট (H2)
```

কেন লেয়ারে ভাগ? প্রতিটি লেয়ারের একটাই কাজ এবং সে শুধু নিচের লেয়ারের সাথে কথা বলে (**separation of concerns**)। এতে প্রতিটি class ছোট, বুঝতে ও টেস্ট করতে সহজ।

লেয়ার	জানে	জানে না
Controller	HTTP, URL, status, DTO	SQL, hashing
Service	নিয়ম, transaction, entity	HTTP
Repository	entity ও query	নিয়ম
Model	কলাম ও টেবিল	বাকি সব

এই কাঠামো Maven-এর স্ট্যান্ডার্ড; Maven ধরে নেয় code `src/main/java`-তে আর Docker ধরে নেয় `Dockerfile` রুট ফোল্ডারে।



চিত্র ১ – লেয়ার। Request উপর থেকে নিচে যায়, response একই পথে ফিরে আসে।

৪. Spring-এর মূল ধারণাগুলো (সব ফাইলেই লাগবে)

- **Bean:** যে object Spring নিজে তৈরি করে ও ম্যানেজ করে। `@Service`, `@RestController`, `@Component`, `@Configuration` দেওয়া class এবং `@Bean` দেওয়া method — সবই bean তৈরি করে। সাধারণত প্রতিটি bean একটাই (singleton)।
- **Dependency Injection (DI):** কোনো class নিজে `new` দিয়ে সাহায্যকারী object বানায় না; constructor-এর parameter হিসেবে চায়, Spring দিয়ে দেয়। যেমন `TeacherService(TeacherRepository r, UserAccountService a)`। সুবিধা: class-গুলো আলগাভাবে যুক্ত, টেস্টে নকল object দেওয়া যায়।
- **Constructor injection** (সব class-এ ব্যবহৃত): field `final` রাখা যায়, dependency স্পষ্ট দেখা যায়, অর্ধেক-তৈরি object কখনো থাকে না। Constructor একটাই হলে `@Autowired` লিখতে হয় না।
- **Component scan:** `@SpringBootApplication` তার নিজের package (`com.university.ums`) ও সব sub-package খুঁজে annotation দেওয়া class পায়। তাই সব class এই package-এর ভেতরে।
- **Auto-configuration:** classpath-এ কোনো লাইব্রেরি (MySQL driver, Spring Security...) দেখলে Spring Boot নিজে থেকেই সেটা সেটআপ করে (DataSource, ObjectMapper, ডিফল্ট security)। আমরা শুধু যা বদলাতে চাই তা লিখি।
- **Annotation:** `@` দিয়ে শুরু হওয়া চিহ্ন যা class/method/field সম্পর্কে framework-কে তথ্য দেয়। নিজে কিছু করে না; Spring বা Hibernate পড়ে কাজ করে।

৫. একটি request-এর পুরো যাত্রা

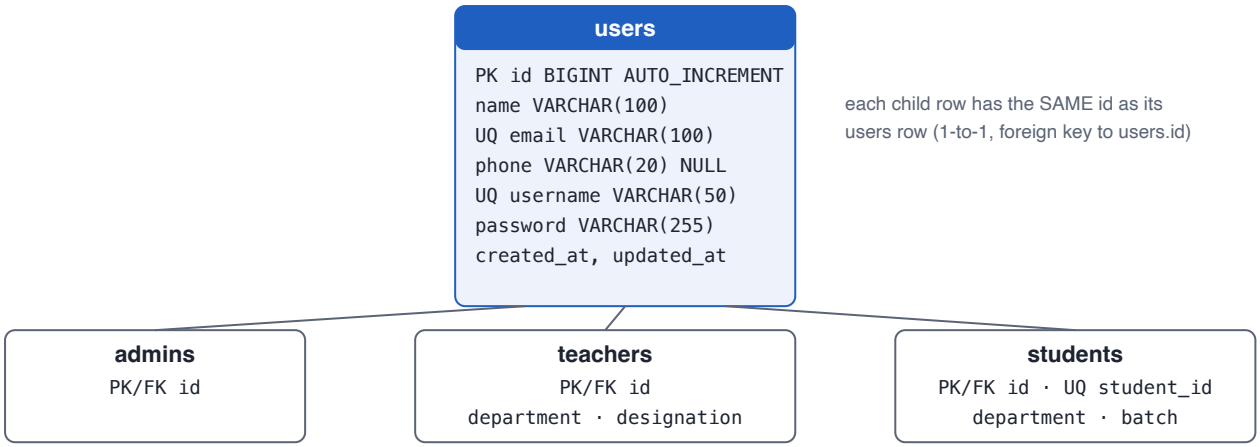
উদাহরণ: লগইন করা একজন **teacher** নতুন teacher বানাতে `POST /api/teachers` পাঠাল।

1. **Tomcat** (jar-এর ভেতরের web server, port 8080) request নেয়।
2. **Security filter**-গুলো প্রথমে চলে: `SecurityContextHolderFilter` `JSESSIONID` cookie দেখে session থেকে লগইন-করা user (ROLE_TEACHER) লোড করে → `LogoutFilter` (এটা logout নয়, তাই এগোয়) → `BasicAuthenticationFilter` (Basic header নেই, এগোয়) → `AuthorizationFilter` নিয়ম মেলায়: `/api/teachers/**`-এ ADMIN বা TEACHER লাগবে → অনুমতি।
3. **DispatcherServlet** খুঁজে বের করে কোন method এই URL-এর জন্য: `TeacherController.create`।
4. **Jackson** JSON body-কে `TeacherRequest` record-এ রূপান্তর করে (ভুল JSON → 400)।
5. **@Valid** annotation-এর নিয়মগুলো যাচাই করে (ভুল হলে → 400, প্রতিটি field-এর বার্তাসহ)।
6. **Controller** → `teacherService.create(request)`।
7. **Service** transaction শুরু করে, password আছে কিনা, username/email আগে নেওয়া কিনা দেখে (→ 409), entity বানায়, BCrypt দিয়ে password hash করে, `save` করে।
8. **Hibernate** চালায় `INSERT INTO users …` ও `INSERT INTO teachers …`; transaction commit হয়।
9. Service entity-কে `TeacherResponse`-এ (password ছাড়া) বদলে ফেরত দেয়।
10. **Controller** নতুন resource-এর URL বানিয়ে **201 Created** + `Location` header ফেরত দেয়; Jackson একে JSON বানায়।

> ধাপ ৪–৯-এ কোনো exception হলে `GlobalExceptionHandler` তা ধরে JSON error পাঠায়। ধাপ ২-এ role না মিললে controller পর্যন্ত যাওয়াই হয় না — `JsonSecurityHandlers` 401/403 JSON লেখে।

৬. ডাটাবেস ডিজাইন

তিন role-এরই অনেক field এক (name, email, phone, username, password)। তিনটা আলাদা টেবিল করলে username তালিকাও তিনটা হতো — একজন teacher আর একজন student দুজনেই `rahim` নিতে পারত, login বুঝত না কে। তাই **inheritance**: abstract parent `AppUser` আর তিনটি child। **JOINED** কৌশলে সাধারণ কলাম `users` টেবিলে, role-নির্দিষ্ট কলাম আলাদা টেবিলে; child row-এর id = users row-এর id (foreign key)।



চিত্র ২ – Hibernate-এর তৈরি টেবিল। PK = primary key, FK = foreign key, UQ = unique।

Java call	যে SQL চলে (সংক্ষেপে)
`teacherRepository.save(t)`	`INSERT INTO users(...)`, তারপর `INSERT INTO teachers(department, designation, id)`
`teacherRepository.findAll()`	`SELECT … FROM teachers t JOIN users u ON u.id = t.id`
`userRepository.findByUsername("karim")`	`users`-এর সাথে তিন child টেবিল LEFT JOIN; যে child টেবিলে row আছে সেই ধরনের object (যেমন `Teacher`) তৈরি হয় — এটাই polymorphism
`teacherRepository.delete(t)`	`DELETE FROM teachers …`, তারপর `DELETE FROM users …`

কৌশল	সুবিধা	অসুবিধা
JOINED (ব্যবহৃত)	normalized, NULL কলাম নেই, সব role মিলিয়ে একটাই unique username তালিকা	পড়তে JOIN লাগে
SINGLE_TABLE	একটাই টেবিল, দ্রুত	child কলাম NULL হতে দিতে হয় (student-এর designation নেই) → দুর্বল integrity
TABLE_PER_CLASS	JOIN নেই	role জুড়ে username unique রাখা ডাটাবেস দিয়ে সম্ভব না

Java field `studentId` → কলাম `student_id`, `createdAt` → `created_at`: Spring Boot camelCase-কে snake_case বানায়। `UK…` / `FK…` নামগুলো Hibernate-এর স্বয়ংক্রিয় constraint নাম।

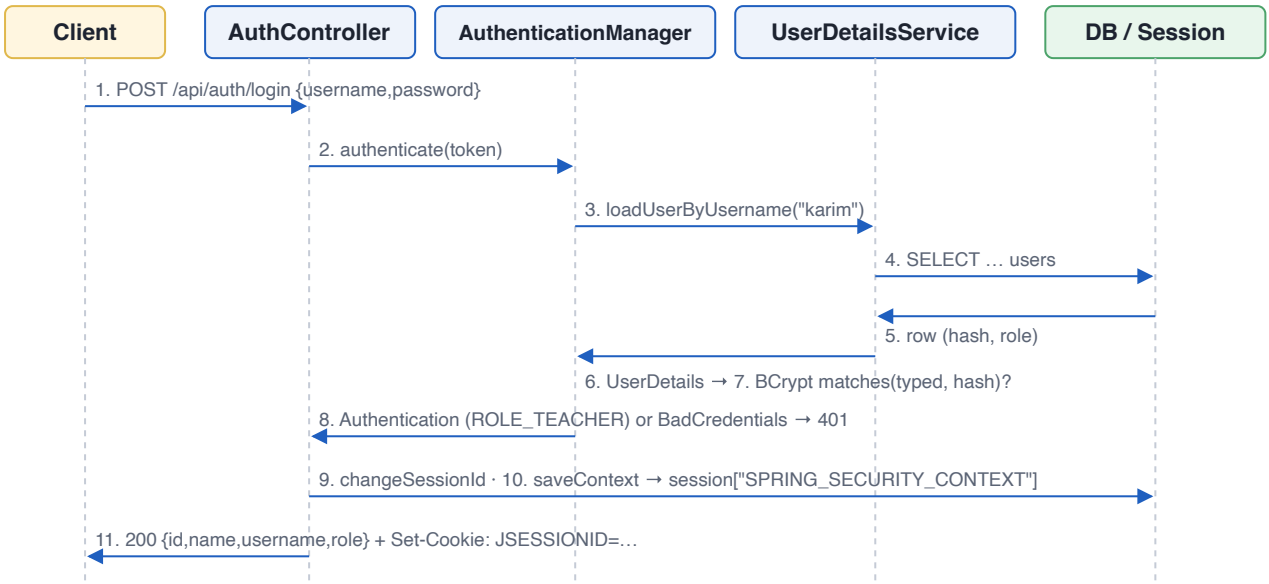
৭. Spring Security-র ধারণা

শব্দ	মানে
Authentication	**তুমি কে** তা প্রমাণ (username + password)। ব্যর্থ হলে 401।
Authorization	**তুমি কী করতে পারো** (role বনাম URL)। ব্যর্থ হলে 403।
Authority / Role	অনুমতির string। Role মানে `ROLE_` prefix-যুক্ত authority: TEACHER role = `ROLE_TEACHER`।
`SecurityContext`	বর্তমান লগইন-করা user-এর `Authentication` ধরে রাখার পাত্র; request চলাকালীন `SecurityContextHolder`-এ থাকে।
Session / `JSESSIONID`	সার্ভারে প্রতিটি client-এর জন্য রাখা জায়গা; তার র‍্যান্ডম id cookie হিসেবে client-এর কাছে যায়।



চিত্র ৩ – প্রধান filter-গুলো ক্রমানুসারে। 401 ও 403 দুটোই আমাদের JsonSecurityHandlers লেখে।

401 না 403 — কীভাবে ঠিক হয়? কেউ লগইন না করলে `AnonymousAuthenticationFilter` একটা "anonymous" user বসায়। `AuthorizationFilter` নিষেধ করলে `AccessDeniedException` ছোড়ে। `ExceptionTranslationFilter` সেটা ধরে: user anonymous হলে সমস্যা "তুমি কে জানি না" → **AuthenticationEntryPoint** → 401। আসল লগইন-করা user হলে সমস্যা "তোমার অনুমতি নেই" → **AccessDeniedHandler** → 403।



চিত্র ৪ – Login-এর ধাপগুলো (বিস্তারিত অধ্যায় ১৫-এ AuthController)।

Login-এর পরের প্রতিটি request

Client নিজে থেকেই `Cookie: JSESSIONID=…` পাঠায় → `SecurityContextHolderFilter` সেই session থেকে `SPRING_SECURITY_CONTEXT` লোড করে → `AuthorizationFilter` role মেলায়। Password আর পাঠাতে হয় না, ডাটাবেসেও যেতে হয় না। ৩০ মিনিট নিষ্ক্রিয় থাকলে session শেষ।

BCrypt দিয়ে password hashing

- **Hashing বনাম encryption:** encryption চাবি দিয়ে উল্টানো যায়; hash **একমুখী**। আসল password আর কখনো বের করা যায় না। Login-এ টাইপ করা password আবার hash করে মেলানো হয়। ডাটাবেস চুরি হলেও শুধু hash পাওয়া যায়।
- **Salt:** প্রতিটি hash-এ র‍্যান্ডম ১৬ বাইট মেশানো হয়, তাই একই password-এর দুটো hash আলাদা, আগে-থেকে-বানানো টেবিল (rainbow table) কাজ করে না।
- **ইচ্ছাকৃত ধীর:** cost factor 10 = 2^{10} = 1024 রাউন্ড। ব্যবহারকারীর কাছে কয়েক মিলিসেকেন্ড, কিন্তু কোটি কোটি অনুমান করা অসম্ভব। MD5/SHA-256 দ্রুত বলেই password-এর জন্য অনুপযুক্ত।
- **গঠন:** `$2a$10$dlWG43JPYk8ap647II8ebO…` → `2a` = সংস্করণ, `10` = cost, পরের ২২ অক্ষর = salt, শেষ ৩১ = hash; মোট ৬০ অক্ষর। Salt hash-এর ভেতরেই থাকে। BCrypt শুধু প্রথম ৭২ বাইট ব্যবহার করে, তাই password সর্বোচ্চ ৭২।

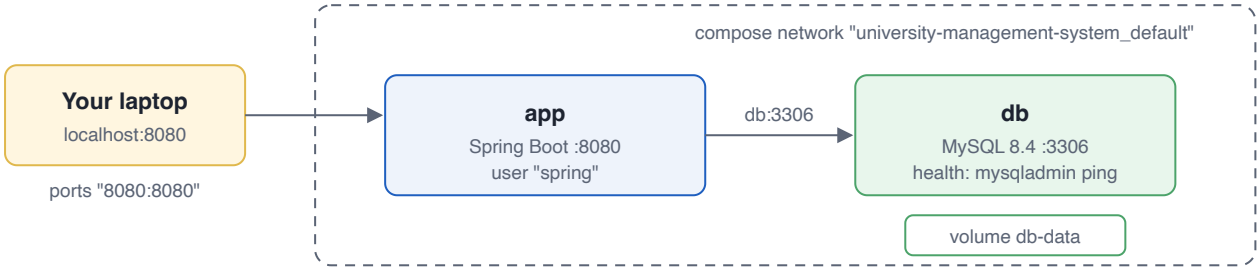
CSRF ও অন্যান্য সিদ্ধান্ত

CSRF বন্ধ কেন, এটা কি নিরাপদ? CSRF মানে: তুমি লগইন থাকা অবস্থায় অন্য কোনো ক্ষতিকর ওয়েবসাইট তোমার ব্রাউজার দিয়ে আমাদের API-তে request পাঠায়, ব্রাউজার cookie নিজে থেকে লাগিয়ে দেয়। Spring-এর ডিফল্ট সুরক্ষা হলো CSRF token, যা HTML form-এর জন্য বানানো; JSON API-তে Postman/curl দিয়ে টেস্ট করা কঠিন করে। তাই token বন্ধ করে session cookie-কে `SameSite=Lax` করা হয়েছে: আধুনিক ব্রাউজার অন্য সাইট থেকে আসা POST/PUT/DELETE-এ cookie পাঠায় না। আমাদের কোনো GET ডাটা বদলায় না, তাই Lax যথেষ্ট। ভবিষ্যতে ব্রাউজার front-end হলে `CookieCsrfTokenRepository` দিয়ে CSRF চালু করা আরও ভালো।

- **HttpOnly cookie:** পেজের JavaScript `JSESSIONID` পড়তে পারে না (XSS দিয়ে চুরি হয় না)।
- **Session fixation:** login-এর সময় session id বদলে দেওয়া হয়, যাতে আক্রমণকারীর আগে থেকে জানা id কাজে না আসে।
- **একই error বার্তা** ভুল username ও ভুল password-এ — কোন username আছে তা জানা যায় না (user enumeration বন্ধ)।
- **HTTP Basic:** প্রতিটি request-এ `Authorization: Basic base64(user:pass)`। Base64 এনকোডিং, এনক্রিপশন নয় — বাস্তবে HTTPS লাগবে। এটা session তৈরি করে না।

৮. Docker-এর ধারণা

শব্দ	মানে
Image	শুধু-পড়া টেমপ্লেট: OS-এর অংশ + Java + আমাদের jar। Dockerfile থেকে বানানো।
Container	image-এর চলমান রূপ — আলাদা filesystem ও network-সহ একটি বিচ্ছিন্ন process।
Layer	Dockerfile-এর প্রতিটি নির্দেশ একটি layer; অপরিবর্তিত layer cache থেকে আসে → দ্রুত rebuild।
Volume	Docker-ম্যানেজড স্টোরেজ, container মুছলেও থাকে (ডাটাবেস ফাইল)।
Network	Compose সব service-কে একটি প্রাইভেট network-এ রাখে; service-এর নামই hostname (`db`)।



চিত্র ৫ – docker compose up যে container-গুলো চালায়। শুধু app-এর port ল্যাপটপে খোলা।

৯. pom.xml — Maven build ফাইল

Maven এই ফাইল পড়ে জানে: কোন লাইব্রেরি নামাতে হবে, কোন Java version-এ compile হবে, আর কীভাবে চালানো যায় এমন jar বানাতে হবে।

pom.xml (86 লাইন)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

লাইন 1–5 XML ঘোষণা ও Maven-এর POM schema। `modelVersion 4.0.0` সব আধুনিক Maven প্রজেক্টে একই।

```xml
    <!-- Spring Boot parent: manages versions of every Spring dependency and plugin -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.16</version>
        <relativePath/>
    </parent>
```

লাইন 7–13 আমাদের প্রজেক্ট `spring-boot-starter-parent 3.5.16`-এর "সন্তান"। Parent সব Spring লাইব্রেরি ও plugin-এর পরীক্ষিত version ঠিক করে দেয় — তাই নিচের dependency-গুলোতে `<version>` লেখা নেই। Parent compiler-এ `-parameters` flag-ও চালু করে, যার কারণে `@PathVariable Long id` নাম ছাড়াই কাজ করে। `relativePath` খালি মানে parent ইন্টারনেট (Maven Central) থেকে নাও।

```xml
    <groupId>com.university</groupId>
    <artifactId>university-management-system</artifactId>
    <version>1.0.0</version>
    <name>University Management System</name>
    <description>CSE 3230 Lab 4 - REST API with Spring Security and Docker</description>
```

লাইন 15–19 প্রজেক্টের পরিচয়: groupId (প্রতিষ্ঠান), artifactId (প্রজেক্টের নাম → jar-এর নাম `university-management-system-1.0.0.jar`), version, name, description।

```xml
    <properties>
        <java.version>21</java.version>
    </properties>
```

লাইন 21–23 `java.version 21` : Java 21-এর জন্য compile হবে।

```xml
    <dependencies>
        <!-- REST controllers, embedded Tomcat, Jackson (JSON) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
```

লাইন 25–30 **Starter** মানে কয়েকটা লাইব্রেরির প্যাকেট। `spring-boot-starter-web` আনে Spring MVC (`@RestController`), embedded Tomcat সার্ভার ও Jackson (JSON)।

```xml
        <!-- JPA + Hibernate for talking to the database -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
```

লাইন 31–35 Spring Data JPA + Hibernate: Java object ডাটাবেসে সেভ/লোড, SQL নিজে তৈরি।

```xml
        <!-- Login, logout, password hashing, role-based access control -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
```

লাইন 36–40 Spring Security: filter chain, login, BCrypt, role check। এটা classpath-এ থাকলেই সব endpoint ডিফল্টভাবে সুরক্ষিত হয়ে যায়।

```
        <!-- @NotBlank, @Email, @Size ... on request bodies -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
```

লাইন 41–45 Hibernate Validator: DTO-র উপর `@NotBlank`, `@Email` ইত্যাদি যাচাই।

```
        <!-- Swagger UI at /swagger-ui.html: browse and call every endpoint from the browser -->
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
```

লাইন 46–51 springdoc: controller-গুলো পড়ে Swagger UI পেজ বানায়। এটা Spring-এর প্রজেক্ট নয়, তাই parent এর version জানে না — একমাত্র dependency যেখানে `<version>2.8.17</version>` নিজে লিখতে হয়েছে (2.8.x লাইন Boot 3.5-এর জন্য)।

```
        <!-- MySQL JDBC driver (used in Docker / production) -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
```

লাইন 52–57 MySQL driver। `scope runtime`: compile-এ লাগে না (আমরা সরাসরি MySQL class ব্যবহার করি না), শুধু চালানোর সময় লাগে।

```
        <!-- In-memory database used only by the automated tests -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

লাইন 59–75 `scope test`: শুধু টেস্টে থাকে, চূড়ান্ত jar-এ যায় না। H2 = মেমরির ডাটাবেস; `spring-boot-starter-test` = JUnit 5, MockMvc, AssertJ; `spring-security-test` = টেস্টে `httpBasic()`, `user()`।

```
    <build>
        <plugins>
            <!-- Packages the app as one executable "fat" jar: java -jar app.jar -->
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

লাইন 77–86 `spring-boot-maven-plugin` সব class ও সব লাইব্রেরি একটা "fat jar"-এ ভরে, যা `java -jar app.jar` দিয়ে চলে। Dockerfile এই jar-ই ব্যবহার করে।

১০. UmsApplication ও model/ — entity (ডাটাবেস টেবিল)

src/main/java/com/university/ums/UmsApplication.java (12 লাইন)

```
package com.university.ums;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

লাইন 1–4 `package`: ফাইলটা কোন ফোল্ডার/নামস্থানে। import: Spring Boot-এর দুটি class।

```java
@SpringBootApplication
public class UmsApplication {

    public static void main(String[] args) {
        SpringApplication.run(UmsApplication.class, args);
    }
}
```

লাইন 6–12 `@SpringBootApplication` = তিনটা annotation একসাথে: `@Configuration` (এই class bean দিতে পারে), `@EnableAutoConfiguration` (লাইব্রেরি দেখে নিজে সেটআপ), `@ComponentScan` (এই package ও নিচের সব annotation-যুক্ত class খোঁজা)। `main` থেকে `SpringApplication.run` পুরো অ্যাপ চালু করে: সব bean তৈরি, ডাটাবেস সংযোগ, Tomcat port 8080-এ চালু, তারপর `CommandLineRunner` (DataInitializer) চালায়।

src/main/java/com/university/ums/model/Role.java (7 লাইন)

```java
package com.university.ums.model;

public enum Role {
    ADMIN,
    TEACHER,
    STUDENT
}
```

লাইন 1–7 `enum` = নির্দিষ্ট কয়েকটা ধ্রুবক মান। String (`"teacher"`) ব্যবহার করলে বানান ভুল ধরা পড়ত না; enum-এ ভুল লিখলে compile error। `Role.TEACHER.name()` দেয় `"TEACHER"`, যা security-তে `ROLE_TEACHER` হয়।

src/main/java/com/university/ums/model/AppUser.java (86 লাইন)

```java
package com.university.ums.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Inheritance;
import jakarta.persistence.InheritanceType;
import jakarta.persistence.PrePersist;
import jakarta.persistence.PreUpdate;
import jakarta.persistence.Table;

import java.time.Instant;
```

লাইন 1–14 package ও import। `jakarta.persistence.*` = JPA-র annotation (আগে `javax.persistence` ছিল; Spring Boot 3 থেকে `jakarta`)। `Instant` = UTC সময়ের একটি মুহূর্ত।

```java
/**
 * Fields shared by every person who can log in (admin, teacher, student).
 * JOINED inheritance: common columns live in "users", role-specific columns
 * live in "admins" / "teachers" / "students", linked by the same id.
 * One "users" table means usernames and emails are unique across all roles.
 */
```

লাইন 16–21 Javadoc মন্তব্য: এই class তিন role-এর সাধারণ field রাখে; JOINED inheritance; একটা `users` টেবিল থাকায় username/email সব role মিলিয়ে unique।

```java
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class AppUser {
```

লাইন 22–25 `@Entity` : এই class একটি টেবিলের সাথে map হবে। `@Table(name = "users")` : টেবিলের নাম (`user` কিছু ডাটাবেসে reserved শব্দ, তাই `users`)। `@Inheritance(JOINED)` : প্রতিটি subclass নিজের টেবিল পাবে, id দিয়ে যুক্ত। `abstract` : সরাসরি `new AppUser()` করা যায় না — প্রত্যেক user অবশ্যই Admin/Teacher/Student। নাম `AppUser` রাখা হয়েছে যাতে Spring Security-র নিজের `User` class-এর সাথে নাম না মেলে।

```
27      @Id
28      @GeneratedValue(strategy = GenerationType.IDENTITY)
29      private Long id;
```

লাইন 27–29 `@Id` : primary key। `@GeneratedValue(IDENTITY)` : id ডাটাবেস নিজে দেয় (MySQL-এর `AUTO_INCREMENT`)। `Long` (wrapper) কারণ নতুন object-এ id `null` থাকে — সেটা দেখেই Hibernate বোঝে INSERT লাগবে। খেয়াল করুন: পরে **setId() নেই**, কেউ id বদলাতে পারে না।

```
31      @Column(nullable = false, length = 100)
32      private String name;
33
34      @Column(nullable = false, unique = true, length = 100)
35      private String email;
36
37      @Column(length = 20)
38      private String phone;
39
40      @Column(nullable = false, unique = true, length = 50)
41      private String username;
```

লাইন 31–41 `@Column` কলামের নিয়ম: `nullable = false` → `NOT NULL` ; `length = 100` → `VARCHAR(100)` ; `unique = true` → UNIQUE index (email ও username)। এটা শেষ প্রতিরক্ষা: Java-র চেক কোনোভাবে বাদ পড়লেও ডাটাবেস ডুপ্লিকেট নেবে না। `phone` -এ nullable নেই, তাই ঐচ্ছিক।

```
43      // BCrypt hash, never the plain-text password
44      @Column(nullable = false)
45      private String password;
```

লাইন 43–45 `password` -এ থাকে BCrypt hash (৬০ অক্ষর), কখনো আসল password নয়। ডিফল্ট দৈর্ঘ্য 255 যথেষ্ট।

```
47      @Column(nullable = false, updatable = false)
48      private Instant createdAt;
49
50      @Column(nullable = false)
51      private Instant updatedAt;
```

লাইন 47–51 `createdAt` -এ `updatable = false` : Hibernate UPDATE-এ এই কলাম কখনো রাখে না, তাই তৈরির সময় একবার বসলে আর বদলায় না। `updatedAt` প্রতিবার বদলায়।

```
53      @PrePersist
54      void onCreate() {
55          createdAt = Instant.now();
56          updatedAt = createdAt;
57      }
58
59      @PreUpdate
60      void onUpdate() {
61          updatedAt = Instant.now();
62      }
```

লাইন 53–62 **JPA lifecycle callback:** `@PrePersist` method Hibernate নিজে ডাকে INSERT-এর ঠিক আগে — দুটো সময় বসায়। `@PreUpdate` প্রতিটি UPDATE-এর আগে `updatedAt` নতুন করে। Service-এ কাউকে সময় বসানো মনে রাখতে হয় না।

```
64      /** Each subclass says which role it is; Spring Security turns this into ROLE_xxx. */
65      public abstract Role getRole();
```

লাইন 64–65 `abstract Role getRole()` : প্রতিটি subclass নিজের role ফেরত দেয়। এটা method, field নয় — তাই `role` নামে কোনো কলাম নেই। Role বোঝা যায় কোন child টেবিলে row আছে তা থেকে, ফলে ডাটার সাথে কখনো অমিল হতে পারে না।

```
67      public Long getId() { return id; }
68
69      public String getName() { return name; }
70      public void setName(String name) { this.name = name; }
71
72      public String getEmail() { return email; }
73      public void setEmail(String email) { this.email = email; }
74
75      public String getPhone() { return phone; }
76      public void setPhone(String phone) { this.phone = phone; }
77
78      public String getUsername() { return username; }
79      public void setUsername(String username) { this.username = username; }
80
81      public String getPassword() { return password; }
82      public void setPassword(String password) { this.password = password; }
83
84      public Instant getCreatedAt() { return createdAt; }
85      public Instant getUpdatedAt() { return updatedAt; }
86  }
```

লাইন 67–86 Getter/setter: JPA ও Jackson এগুলো দিয়ে field পড়ে/লেখে। Lombok ব্যবহার করা হয়নি যাতে লুকানো generated code ব্যাখ্যা করতে না হয়। `getId`, `getCreatedAt`, `getUpdatedAt`-এর setter নেই — বাইরের কেউ বদলাতে পারে না।

src/main/java/com/university/ums/model/Teacher.java (28 লাইন)

```
1  package com.university.ums.model;
2
3  import jakarta.persistence.Column;
4  import jakarta.persistence.Entity;
5  import jakarta.persistence.Table;
```

লাইন 1–5 import: JPA annotation।

```
7  @Entity
8  @Table(name = "teachers")
9  public class Teacher extends AppUser {
```

লাইন 7–9 `@Entity @Table(name = "teachers")` : নিজস্ব টেবিল। `extends AppUser` : Teacher *একজন* AppUser — id, name, email, username, password, সময় সব উত্তরাধিকারে পায়। JPA-র জন্য argument-ছাড়া constructor লাগে; অন্য constructor না থাকায় Java নিজে দেয়।

```
11      @Column(nullable = false, length = 100)
12      private String department;
13
14      // e.g. Lecturer, Assistant Professor, Professor
15      @Column(nullable = false, length = 50)
16      private String designation;
```

লাইন 11–16 Teacher-এর নিজস্ব দুটি কলাম: `department` (VARCHAR 100) ও `designation` (Lecturer, Professor...; VARCHAR 50), দুটোই NOT NULL।

```
18      @Override
19      public Role getRole() {
20          return Role.TEACHER;
21      }
```

লাইন 18–21 `@Override getRole()` → `Role.TEACHER`। `@Override` compiler-কে বলে "এটা parent-এর method বাস্তবায়ন করছে" — নাম ভুল হলে error।

```
23      public String getDepartment() { return department; }
24      public void setDepartment(String department) { this.department = department; }
25
26      public String getDesignation() { return designation; }
27      public void setDesignation(String designation) { this.designation = designation; }
28  }
```

লাইন 23–28 দুই field-এর getter/setter।

Admin.java-এ কোনো বাড়তি কলাম নেই: শুধু `@Entity @Table(name = "admins")`, `extends AppUser` আর `getRole()` → `Role.ADMIN`। তার টেবিলে শুধু id থাকে — ওই row-টাই বলে দেয় এই user admin।

src/main/java/com/university/ums/model/Student.java (35 লাইন)

```java
package com.university.ums.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;
```

লাইন 1–5 import।

```java
@Entity
@Table(name = "students")
public class Student extends AppUser {
```

লাইন 7–9 `students` টেবিল, `extends AppUser`।

```java
    // University roll / registration number, e.g. "2007001"
    @Column(nullable = false, unique = true, length = 20)
    private String studentId;
```

লাইন 11–13 `studentId` = রোল/রেজিস্ট্রেশন নম্বর, `unique = true` → দুজন student-এর একই রোল হতে পারে না। কলামের নাম হবে `student_id`।

```java
    @Column(nullable = false, length = 100)
    private String department;
```

লাইন 15–16 বিভাগ (department)।

```java
    // Admission year, e.g. 2021
    @Column(nullable = false)
    private Integer batch;
```

লাইন 18–20 `batch` = ভর্তির বছর। `Integer` (wrapper) কারণ JSON-এ না এলে `null` হয়, আর validation সেটা ধরতে পারে; `int` হলে চুপচাপ 0 হয়ে যেত।

```java
    @Override
    public Role getRole() {
        return Role.STUDENT;
    }
```

লাইন 22–25 `getRole()` → `Role.STUDENT`।

```java
    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    public Integer getBatch() { return batch; }
    public void setBatch(Integer batch) { this.batch = batch; }
}
```

লাইন 27–35 Getter/setter।

১১. repository/ — Spring Data দিয়ে ডাটাবেস অ্যাক্সেস

কোনো code ছাড়া interface কীভাবে কাজ করে? চালু হওয়ার সময় Spring Data প্রতিটি interface-এর জন্য একটা class (proxy) নিজে বানিয়ে bean হিসেবে রাখে। `JpaRepository<Teacher, Long>` মানে "entity Teacher, id-এর ধরন Long", আর বিনামূল্যে দেয়: `save`, `saveAndFlush`, `findById` (ফেরত দেয় `Optional`), `findAll`, `count`, `delete`, `existsById`। **Derived query:** method-এর নাম পড়েই query বানায় — `findByUsername` → `WHERE username = ?`।

src/main/java/com/university/ums/repository/UserRepository.java (21 লাইন)

```java
package com.university.ums.repository;

import com.university.ums.model.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;
```

লাইন 1–6 import।

```
/** Queries across all three roles at once (the shared "users" table). */
public interface UserRepository extends JpaRepository<AppUser, Long> {
```

লাইন 8–9 `AppUser` (parent)-এর repository, তাই এর query **তিন role একসাথে** খোঁজে। Login ও uniqueness চেক এটা ব্যবহার করে।

```
    Optional<AppUser> findByUsername(String username);
```

লাইন 11 `findByUsername` → `SELECT … WHERE username = ?`। ফেরত `Optional`: পাওয়া গেলে ভেতরে user, না পেলে খালি — কখনো `null` নয়, তাই NullPointerException-এর ঝুঁকি কম। Login (`CustomUserDetailsService`) ও `/api/auth/me` এটা ব্যবহার করে।

```
    boolean existsByUsername(String username);

    boolean existsByEmail(String email);
```

লাইন 13–15 "এই username/email-এর অন্তত একটা row আছে?" → `true/false`। নতুন user তৈরির সময় ব্যবহৃত।

```
    // "...AndIdNot": ignore the user being updated, so keeping your own username is not a conflict
    boolean existsByUsernameAndIdNot(String username, Long id);

    boolean existsByEmailAndIdNot(String email, Long id);
}
```

লাইন 17–21 `existsByUsernameAndIdNot(u, id)` → `WHERE username = ? AND id <> ?`। **কেন?** Teacher #5 আপডেটের সময় নিজের username `karim` রাখলে সাধারণ `existsByUsername` teacher #5-কেই খুঁজে পেয়ে ভুল করে "ডুপ্লিকেট" বলত। নিজের id বাদ দিয়ে প্রশ্ন হয় "*অন্য কারো* কি এই username আছে?"

src/main/java/com/university/ums/repository/StudentRepository.java (11 লাইন)

```
package com.university.ums.repository;

import com.university.ums.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
```

লাইন 1–4 import।

```
public interface StudentRepository extends JpaRepository<Student, Long> {
```

লাইন 6 `JpaRepository<Student, Long>` থেকে সব CRUD method। `AdminRepository` ও `TeacherRepository` হুবহু এরকম, শুধু ভেতরে কোনো method নেই (ফাইলে মাত্র কয়েক লাইন)।

```
    boolean existsByStudentId(String studentId);

    boolean existsByStudentIdAndIdNot(String studentId, Long id);
}
```

লাইন 8–11 রোল নম্বর ডুপ্লিকেট কিনা — create-এ `existsByStudentId`, update-এ নিজেকে বাদ দিয়ে `existsByStudentIdAndIdNot`। `studentId` property → `student_id` কলাম।

১২. dto/ — JSON-এর আকার

DTO (Data Transfer Object) শুধু API-তে ডাটা আনা-নেওয়ার জন্য। Entity সরাসরি পাঠানো/নেওয়া হয় না, কারণ: (১) response DTO-তে password field নেই → hash কখনো ফাঁস হয় না; (২) request DTO-তে শুধু অনুমোদিত field → client `"id"`, `"role":"ADMIN"`, `"createdAt"` পাঠিয়ে বদলাতে পারে না (**mass assignment** বন্ধ); (৩) validation API-র ইনপুটে থাকে; (৪) ডাটাবেস বদলালেও API একই থাকে।

Java record: `record TeacherRequest(String name, …)` একটা অপরিবর্তনীয় (immutable) ডাটা class-এর ছোট রূপ। Java নিজে বানায়: constructor, `name()`, `email()`… accessor, `equals`, `hashCode`, `toString`। Jackson JSON ↔ record দুদিকেই পারে।

src/main/java/com/university/ums/dto/TeacherRequest.java (23 লাইন)

```
package com.university.ums.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
```

লাইন 1–6 Validation annotation-গুলোর import (`jakarta.validation.constraints`)।

```
 8 /**
 9  * JSON body for POST /api/teachers and PUT /api/teachers/{id}.
10  * password is required on create, optional on update (omit it to keep the old one).
11  */
```

লাইন 8–11 `POST /api/teachers` ও `PUT /api/teachers/{id}`-এর JSON body। password create-এ বাধ্যতামূলক, update-এ ঐচ্ছিক।

```
12 public record TeacherRequest(
```

লাইন 12 record শুরু; প্রতিটি component JSON-এর একটা key (`"name"`, `"email"` ...)।

```
13         @NotBlank @Size(max = 100) String name,
```

লাইন 13 `@NotBlank`: null, "" বা শুধু space হলে চলবে না। `@Size(max = 100)`: কলামের দৈর্ঘ্যের সমান, যাতে ডাটাবেস কখনো লম্বা মান প্রত্যাখ্যান না করে।

```
14         @NotBlank @Email @Size(max = 100) String email,
```

লাইন 14 `@Email`: ইমেইলের মতো দেখতে হতে হবে (না হলে বার্তা "must be a well-formed email address")।

```
15         @Size(max = 20) String phone,
```

লাইন 15 `phone`-এ `@NotBlank` নেই → ঐচ্ছিক। `null` মানে `@Size` পাস করে।

```
16         @NotBlank @Size(min = 3, max = 50)
17         @Pattern(regexp = "^[A-Za-z0-9._-]+$", message = "may contain only letters, digits, '.', '_' and '-'")
18         String username,
```

লাইন 16–18 username: ৩–৫০ অক্ষর, আর `@Pattern` regex `^[A-Za-z0-9._-]+$` — শুধু অক্ষর, সংখ্যা, `.` `_` `-`; space নয়। `message` দিয়ে নিজস্ব error বার্তা।

```
19         @Size(min = 6, max = 72) String password,
```

লাইন 19 password-এ `@NotBlank` নেই কারণ একই DTO create ও update-এ ব্যবহৃত। Create-এ service নিজে চায় (`requirePassword` → 400); update-এ না পাঠালে আগের password থাকে। সর্বোচ্চ ৭২ কারণ BCrypt শুধু প্রথম ৭২ বাইট নেয়।

```
20         @NotBlank @Size(max = 100) String department,
21         @NotBlank @Size(max = 50) String designation
22 ) {
23 }
```

লাইন 20–23 Teacher-এর নিজস্ব দুটি বাধ্যতামূলক field। এই annotation-গুলো নিজে কিছু করে না; controller-এ `@Valid` থাকলে তবেই যাচাই হয়।

AdminRequest একই রকম, শুধু প্রথম পাঁচটি field (department/designation নেই)।

src/main/java/com/university/ums/dto/StudentRequest.java (27 লাইন)

```
 1 package com.university.ums.dto;
 2
 3 import jakarta.validation.constraints.Email;
 4 import jakarta.validation.constraints.Max;
 5 import jakarta.validation.constraints.Min;
 6 import jakarta.validation.constraints.NotBlank;
 7 import jakarta.validation.constraints.NotNull;
 8 import jakarta.validation.constraints.Pattern;
 9 import jakarta.validation.constraints.Size;
10
11 /**
12  * JSON body for POST /api/students and PUT /api/students/{id}.
13  * password is required on create, optional on update (omit it to keep the old one).
14  */
```

লাইন 1–14 `@Min`, `@Max`, `@NotNull`-এর import বাড়তি।

```
15 public record StudentRequest(
16         @NotBlank @Size(max = 100) String name,
17         @NotBlank @Email @Size(max = 100) String email,
18         @Size(max = 20) String phone,
19         @NotBlank @Size(min = 3, max = 50)
20         @Pattern(regexp = "^[A-Za-z0-9._-]+$", message = "may contain only letters, digits, '.', '_' and '-'")
21         String username,
```

লাইন 15–21 সাধারণ পাঁচটি field TeacherRequest-এর মতোই।

```
22         @Size(min = 6, max = 72) String password,
```

লাইন 22 রোল নম্বর বাধ্যতামূলক, সর্বোচ্চ ২০ (কলামের সমান); বিভাগ বাধ্যতামূলক।

```
23         @NotBlank @Size(max = 20) String studentId,
24         @NotBlank @Size(max = 100) String department,
25         @NotNull @Min(1950) @Max(2100) Integer batch
26 ) {
27 }
```

লাইন 23–27 `batch` একটা `Integer` — String নয় বলে `@NotBlank` চলে না, তাই `@NotNull`। `@Min(1950) @Max(2100)` : যুক্তিসঙ্গত বছর। JSON-এ `"batch":"abc"` দিলে Jackson-ই ব্যর্থ হয় → 400 "Malformed JSON"।

src/main/java/com/university/ums/dto/TeacherResponse.java (33 লাইন)

```
1 package com.university.ums.dto;
2
3 import com.university.ums.model.Role;
4 import com.university.ums.model.Teacher;
5
6 import java.time.Instant;
```

লাইন 1–6 import।

```
8 public record TeacherResponse(
9         Long id,
10        String name,
11        String email,
12        String phone,
13        String username,
14        Role role,
15        String department,
16        String designation,
17        Instant createdAt,
18        Instant updatedAt
19 ) {
```

লাইন 8–19 API যা ফেরত পাঠায়। **password field নেই**। `role` ও সময়গুলো আছে।

```
20     public static TeacherResponse from(Teacher teacher) {
21         return new TeacherResponse(
22                 teacher.getId(),
23                 teacher.getName(),
24                 teacher.getEmail(),
25                 teacher.getPhone(),
26                 teacher.getUsername(),
27                 teacher.getRole(),
28                 teacher.getDepartment(),
29                 teacher.getDesignation(),
30                 teacher.getCreatedAt(),
31                 teacher.getUpdatedAt());
32     }
33 }
```

লাইন 20–33 `static from(Teacher)` : entity থেকে DTO-তে কপি করার **factory/mapper** method। Service সবসময় DTO ফেরত দেয়, তাই controller কখনো entity ছোঁয় না। `AdminResponse` ও `StudentResponse` হুবহু এই নকশায়, নিজ নিজ field নিয়ে। `Instant` JSON-এ ISO-8601 রূপে যায় (শেষে `Z` = UTC)।

src/main/java/com/university/ums/dto/LoginRequest.java (11 লাইন)

```java
package com.university.ums.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
```

লাইন 1–4 import: Swagger-এর `@Schema` ও validation।

```java
public record LoginRequest(
        // example values pre-fill the Swagger UI form
        @Schema(example = "admin") @NotBlank String username,
        @Schema(example = "admin123") @NotBlank String password
) {
}
```

লাইন 6–11 Login-এর body: `{"username": "...", "password": "..."}`। দুটোই `@NotBlank`। `@Schema(example = "admin")` শুধু ডকুমেন্টেশন — Swagger UI-র form আগে থেকে ভরে দেয়; validation বা security-তে কোনো প্রভাব নেই।

src/main/java/com/university/ums/dto/CurrentUserResponse.java (16 লাইন)

```java
package com.university.ums.dto;

import com.university.ums.model.AppUser;
import com.university.ums.model.Role;
```

লাইন 1–4 import।

```java
/** Returned by /api/auth/login and /api/auth/me: who is logged in and with which role. */
public record CurrentUserResponse(
        Long id,
        String name,
        String username,
        Role role
) {
    public static CurrentUserResponse from(AppUser user) {
        return new CurrentUserResponse(user.getId(), user.getName(), user.getUsername(), user.getRole());
    }
}
```

লাইন 6–16 Login ও `/api/auth/me`-এর উত্তর: কে লগইন, কোন role-এ। `from(AppUser)` যেকোনো role-এর user নেয় (polymorphism: `user.getRole()` আসল subclass-এর method চালায়)। `MessageResponse(String message)` একই রকম ছোট record, logout-এর উত্তরে ব্যবহৃত।

১৩. exception/ — একই ধরনের JSON error

src/main/java/com/university/ums/exception/ApiError.java (21 লাইন)

```java
package com.university.ums.exception;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.time.Instant;
import java.util.Map;
```

লাইন 1–6 Jackson-এর `@JsonInclude` import।

```java
/** One JSON shape for every error the API returns. fieldErrors only appears on validation errors. */
```

লাইন 8 API-র প্রতিটি error একই আকারে আসে, তাই client সবসময় জানে বার্তা কোথায় পাবে।

```java
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ApiError(
        Instant timestamp,
        int status,
        String error,
        String message,
        String path,
        Map<String, String> fieldErrors
) {
```

লাইন 9–17 `@JsonInclude(NON_NULL)`: যে field `null` তা JSON-এ লেখা হবে না — validation error ছাড়া `fieldErrors` দেখা যায় না। Record-এর field: সময়, status নম্বর, status-এর নাম, বার্তা, URL path, আর field-ভিত্তিক error-এর Map।

```
18     public static ApiError of(int status, String error, String message, String path) {
19         return new ApiError(Instant.now(), status, error, message, path, null);
20     }
21 }
```

লাইন 18–21 সুবিধার factory method: fieldErrors ছাড়া ApiError বানায়, সময় এখনকার।

src/main/java/com/university/ums/exception/ResourceNotFoundException.java (9 লাইন)

```
1 package com.university.ums.exception;
```

লাইন 1 package।

```
3 /** Mapped to 404 Not Found by GlobalExceptionHandler. */
4 public class ResourceNotFoundException extends RuntimeException {
5
6     public ResourceNotFoundException(String resource, Long id) {
7         super(resource + " with id " + id + " not found");
8     }
9 }
```

লাইন 3–9 `RuntimeException` থেকে extend (unchecked): method-এ `throws` লিখতে হয় না, আর ছুড়লে `@Transactional` নিজে rollback করে। Constructor বার্তা বানায়: "Teacher with id 5 not found"। `ConflictException` (409) ও `BadRequestException` (400) হুবহু একই, শুধু সরাসরি বার্তা নেয়।

src/main/java/com/university/ums/exception/GlobalExceptionHandler.java (81 লাইন)

```
1 package com.university.ums.exception;
2
3 import jakarta.servlet.http.HttpServletRequest;
4 import org.springframework.dao.DataIntegrityViolationException;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.http.converter.HttpMessageNotReadableException;
8 import org.springframework.security.core.AuthenticationException;
9 import org.springframework.validation.FieldError;
10 import org.springframework.web.bind.MethodArgumentNotValidException;
11 import org.springframework.web.bind.annotation.ExceptionHandler;
12 import org.springframework.web.bind.annotation.RestControllerAdvice;
13 import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;
14
15 import java.time.Instant;
16 import java.util.LinkedHashMap;
17 import java.util.Map;
```

লাইন 1–17 import: servlet request, Spring-এর বিভিন্ন exception, Security-র `AuthenticationException`।

```
19 /**
20  * Turns exceptions thrown by controllers/services into JSON error responses
21  * with the right HTTP status code, so controllers stay free of try/catch.
22  */
```

লাইন 19–22 উদ্দেশ্য: controller/service থেকে ছোড়া exception-কে সঠিক status-সহ JSON-এ বদলানো, যাতে controller-এ try/catch না লাগে।

```
23 @RestControllerAdvice
24 public class GlobalExceptionHandler {
```

লাইন 23–24 `@RestControllerAdvice`: সব controller-এর জন্য একটা global সাহায্যকারী। কোনো controller (বা তার ডাকা service) exception ছুড়লে Spring এখানে মিলিয়ে দেখে কোন `@ExceptionHandler` method সেই ধরন ধরে, সেটা ডাকে এবং তার return-কে response বানায়।

```
26     @ExceptionHandler(ResourceNotFoundException.class)
27     public ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex, HttpServletRequest request) {
28         return build(HttpStatus.NOT_FOUND, ex.getMessage(), request);
29     }
```

লাইন 26–29 `ResourceNotFoundException` → 404। Method parameter-এ exception ও request দুটোই Spring দেয় (request থেকে path নেওয়া হয়)।

```
31      @ExceptionHandler(ConflictException.class)
32      public ResponseEntity<ApiError> handleConflict(ConflictException ex, HttpServletRequest request) {
33          return build(HttpStatus.CONFLICT, ex.getMessage(), request);
34      }
```

লাইন 31–34 `ConflictException` → 409 Conflict।

```
36      @ExceptionHandler(BadRequestException.class)
37      public ResponseEntity<ApiError> handleBadRequest(BadRequestException ex, HttpServletRequest request) {
38          return build(HttpStatus.BAD_REQUEST, ex.getMessage(), request);
39      }
```

লাইন 36–39 `BadRequestException` → 400।

```
41      // @Valid failed: collect every invalid field and its message
42      @ExceptionHandler(MethodArgumentNotValidException.class)
43      public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex, HttpServletRequest request)
{
44          Map<String, String> fieldErrors = new LinkedHashMap<>();
45          for (FieldError error : ex.getBindingResult().getFieldErrors()) {
46              fieldErrors.putIfAbsent(error.getField(), error.getDefaultMessage());
47          }
48          ApiError body = new ApiError(Instant.now(), 400, "Bad Request", "Validation failed",
49                  request.getRequestURI(), fieldErrors);
50          return ResponseEntity.badRequest().body(body);
51      }
```

লাইন 41–51 `@Valid` ব্যর্থ → `MethodArgumentNotValidException`। সব ভুল field-এর উপর loop চালিয়ে `field → বার্তা` একটা `LinkedHashMap`-এ রাখা হয় (ঢোকানোর ক্রম ঠিক থাকে)। `putIfAbsent`: একটা field-এর একাধিক ভুল হলে প্রথমটাই রাখে। ফলাফল 400 + `fieldErrors`।

```
53      // Body is not valid JSON, or a field has the wrong type (e.g. "batch": "abc")
54      @ExceptionHandler(HttpMessageNotReadableException.class)
55      public ResponseEntity<ApiError> handleUnreadable(HttpMessageNotReadableException ex, HttpServletRequest request)
{
56          return build(HttpStatus.BAD_REQUEST, "Malformed JSON request body", request);
57      }
```

লাইন 53–57 Body বৈধ JSON না, বা field-এর ধরন ভুল → Jackson ব্যর্থ → 400 "Malformed JSON request body"।

```
59      // Path variable has the wrong type, e.g. /api/students/abc
60      @ExceptionHandler(MethodArgumentTypeMismatchException.class)
61      public ResponseEntity<ApiError> handleTypeMismatch(MethodArgumentTypeMismatchException ex, HttpServletRequest
request) {
62          return build(HttpStatus.BAD_REQUEST, "Invalid value '" + ex.getValue() + "' for parameter '" + ex.getName()
+ "'", request);
63      }
```

লাইন 59–63 `/api/students/abc`: "abc"-কে `Long` বানানো যায় না → 400, বার্তায় কোন মান ও কোন parameter।

```
65      // Safety net: the database rejected a duplicate the service checks did not catch (e.g. two requests at once)
66      @ExceptionHandler(DataIntegrityViolationException.class)
67      public ResponseEntity<ApiError> handleIntegrity(DataIntegrityViolationException ex, HttpServletRequest request)
{
68          return build(HttpStatus.CONFLICT, "Request conflicts with existing data (duplicate value)", request);
69      }
```

লাইন 65–69 **Race condition-এর নিরাপত্তা জাল:** একই username নিয়ে দুটো request একই মুহূর্তে এলে দুটোরই Java চেক পাস করতে পারে, কিন্তু ডাটাবেসের UNIQUE constraint দ্বিতীয় INSERT ফিরিয়ে দেয় → `DataIntegrityViolationException` → 409।

```
71      // Wrong username/password on POST /api/auth/login
72      @ExceptionHandler(AuthenticationException.class)
73      public ResponseEntity<ApiError> handleAuthentication(AuthenticationException ex, HttpServletRequest request) {
74          return build(HttpStatus.UNAUTHORIZED, "Invalid username or password", request);
75      }
```

লাইন 71–75 Login-এ ভুল credential → `BadCredentialsException` (এক ধরনের `AuthenticationException`) → 401। বার্তা ইচ্ছাকৃতভাবে অস্পষ্ট: username ভুল না password ভুল বলা হয় না, যাতে কোন username আছে তা জানা না যায়।

```
77      private ResponseEntity<ApiError> build(HttpStatus status, String message, HttpServletRequest request) {
78          ApiError body = ApiError.of(status.value(), status.getReasonPhrase(), message, request.getRequestURI());
79          return ResponseEntity.status(status).body(body);
80      }
81  }
```

লাইন 77–81 সাহায্যকারী: status ও বার্তা থেকে একই রকম `ApiError` বানিয়ে `ResponseEntity` (status + body) ফেরত দেয়। `getReasonPhrase()` → "Not Found", "Conflict" ইত্যাদি।

Code	মানে	কখন
200 / 201 / 204	সফল / তৈরি হয়েছে / সফল কিন্তু body নেই	GET, PUT, login / POST / DELETE
400	client ভুল ডাটা পাঠিয়েছে	validation, ভুল JSON, id সংখ্যা না, create-এ password নেই
401	authenticate হয়নি (কে জানি না)	লগইন নেই/session শেষ, ভুল password
403	লগইন আছে কিন্তু অনুমতি নেই	teacher → /api/students
404 / 409	নেই / বর্তমান অবস্থার সাথে সংঘাত	অজানা id / ডুপ্লিকেট, শেষ admin মোছা

১৪. service/ — ব্যবসায়িক নিয়ম ও transaction

src/main/java/com/university/ums/service/UserAccountService.java (69 লাইন)

```
1  package com.university.ums.service;
2
3  import com.university.ums.exception.BadRequestException;
4  import com.university.ums.exception.ConflictException;
5  import com.university.ums.model.AppUser;
6  import com.university.ums.repository.UserRepository;
7  import org.springframework.security.crypto.password.PasswordEncoder;
8  import org.springframework.stereotype.Service;
9
10 import java.util.Locale;
```

লাইন 1–10 import: exception, entity, repository, `PasswordEncoder`, `Locale`।

```
12 /** Login-account logic shared by the admin, teacher and student services. */
13 @Service
14 public class UserAccountService {
```

লাইন 12–14 **কেন আলাদা shared service?** তিন role-এরই একই তিনটা কাজ লাগে: uniqueness চেক, field কপি, password hash। একবার লিখলে তিন কপির মধ্যে অমিলের ঝুঁকি নেই। `@Service` → এটা একটা bean।

```
16      private final UserRepository userRepository;
17      private final PasswordEncoder passwordEncoder;
18
19      public UserAccountService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
20          this.userRepository = userRepository;
21          this.passwordEncoder = passwordEncoder;
22      }
```

লাইন 16–22 `private final` dependency + constructor injection: Spring `UserRepository` ও `PasswordEncoder` (SecurityConfig-এর BCrypt bean) পাঠায়।

```
24      /** Usernames and emails are stored lower-case so "Rahim" and "rahim" are the same account. */
25      public static String normalize(String value) {
26          return value.trim().toLowerCase(Locale.ROOT);
27      }
```

লাইন 24–27 `normalize()`: সামনে-পেছনের space কাটে, ছোট হাতের করে। `Locale.ROOT`: সব কম্পিউটারে একই ফল (তুর্কি locale-এ `"I".toLowerCase()` "i" হয় না)। ফলে `Rahim`, `RAHIM`, `rahim` একই account — MySQL (case-insensitive) ও H2 (case-sensitive) দুটোতেই একই আচরণ। `static` কারণ object-এর কোনো ডাটা লাগে না; `CustomUserDetailsService`-ও ব্যবহার করে।

```
29      public void requirePassword(String rawPassword) {
30          if (rawPassword == null || rawPassword.isBlank()) {
31              throw new BadRequestException("Password is required when creating a user");
32          }
33      }
```

লাইন 29–33 Create-এ password নেই বা ফাঁকা → `BadRequestException` → 400।

```
35      /**
36       * Throws 409 if the username or email already belongs to someone else (in any role).
37       * currentUserId is null on create, or the id of the user being updated.
38       */
39      public void ensureUnique(String username, String email, Long currentUserId) {
```

লাইন 35–39 `ensureUnique(username, email, currentUserId)`-এর ব্যাখ্যা: `currentUserId == null` মানে তৈরি হচ্ছে, না হলে আপডেট হচ্ছে।

```
40          String normalizedUsername = normalize(username);
41          String normalizedEmail = normalize(email);
42
43          boolean usernameTaken = currentUserId == null
44                  ? userRepository.existsByUsername(normalizedUsername)
45                  : userRepository.existsByUsernameAndIdNot(normalizedUsername, currentUserId);
46          if (usernameTaken) {
47              throw new ConflictException("Username '" + normalizedUsername + "' is already taken");
48          }
```

লাইন 40–48 তুলনার আগে normalize। `boolean usernameTaken = শর্ত ? ক : খ` — ternary operator: create হলে `existsByUsername`, update হলে নিজেকে বাদ দিয়ে `existsByUsernameAndIdNot`। নেওয়া থাকলে `ConflictException` → 409 "Username 'rahim' is already taken"।

```
50          boolean emailTaken = currentUserId == null
51                  ? userRepository.existsByEmail(normalizedEmail)
52                  : userRepository.existsByEmailAndIdNot(normalizedEmail, currentUserId);
53          if (emailTaken) {
54              throw new ConflictException("Email '" + normalizedEmail + "' is already in use");
55          }
56      }
```

লাইন 50–56 Email-এর জন্য হুবহু একই যুক্তি।

```
58      /** Copies the common fields onto the entity. The password is hashed with BCrypt, never stored raw. */
59      public void applyCommonFields(AppUser user, String name, String email, String phone,
60                                    String username, String rawPassword) {
```

লাইন 58–60 `applyCommonFields(AppUser user, …)`: parameter-এর ধরন `AppUser`, তাই Admin, Teacher, Student যেকোনোটা দেওয়া যায় (inheritance)।

```
61          user.setName(name.trim());
62          user.setEmail(normalize(email));
63          user.setPhone(phone == null || phone.isBlank() ? null : phone.trim());
64          user.setUsername(normalize(username));
```

লাইন 61–64 name trim, email ও username normalize; phone ফাঁকা হলে `null` রাখা হয়।

```
65          if (rawPassword != null && !rawPassword.isBlank()) {
66              user.setPassword(passwordEncoder.encode(rawPassword));
67          }
68      }
69  }
```

লাইন 65–69 **Password তখনই বদলায় যখন নতুন দেওয়া হয়**, এবং `passwordEncoder.encode()` দিয়ে BCrypt hash হয়ে যায় — আসল password কোথাও সেভ হয় না। Update-এ password না পাঠালে এই if-এ ঢোকে না, পুরনো hash থাকে।

src/main/java/com/university/ums/service/AdminService.java (72 লাইন)

```
1  package com.university.ums.service;
2
3  import com.university.ums.dto.AdminRequest;
4  import com.university.ums.dto.AdminResponse;
5  import com.university.ums.exception.ConflictException;
6  import com.university.ums.exception.ResourceNotFoundException;
7  import com.university.ums.model.Admin;
8  import com.university.ums.repository.AdminRepository;
9  import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.util.List;
```

লাইন 1–12 import।

```
@Service
@Transactional
public class AdminService {
```

লাইন 14–16 `@Service`: bean। `@Transactional` **class-এর উপর** → প্রতিটি public method একটা ডাটাবেস transaction-এ চলে: method-এর আগে শুরু, স্বাভাবিকভাবে শেষ হলে **commit**, `RuntimeException` ছুড়লে **rollback**। মানে হয় পুরো কাজ হবে, নয়তো কিছুই না (atomicity)। Spring এটা করে bean-কে একটা proxy-তে মুড়ে।

```
    private final AdminRepository adminRepository;
    private final UserAccountService accounts;

    public AdminService(AdminRepository adminRepository, UserAccountService accounts) {
        this.adminRepository = adminRepository;
        this.accounts = accounts;
    }
```

লাইন 18–24 Dependency ও constructor injection।

```
    @Transactional(readOnly = true)
    public List<AdminResponse> findAll() {
        return adminRepository.findAll().stream().map(AdminResponse::from).toList();
    }
```

লাইন 26–29 `findAll()`: সব admin লোড → `.stream().map(AdminResponse::from).toList()`: প্রতিটিকে DTO-তে বদলে list। `AdminResponse::from` = method reference, অর্থাৎ `a -> AdminResponse.from(a)`। `readOnly = true`: Hibernate পরিবর্তন ট্র্যাক করে না — ছোট অপ্টিমাইজেশন ও উদ্দেশ্য স্পষ্ট।

```
    @Transactional(readOnly = true)
    public AdminResponse findById(Long id) {
        return AdminResponse.from(getOrThrow(id));
    }
```

লাইন 31–34 `findById`: `getOrThrow` দিয়ে খুঁজে DTO; না পেলে 404।

```
    public AdminResponse create(AdminRequest request) {
        accounts.requirePassword(request.password());
        accounts.ensureUnique(request.username(), request.email(), null);

        Admin admin = new Admin();
        apply(admin, request);
        return AdminResponse.from(adminRepository.save(admin));
    }
```

লাইন 36–43 **ক্রম গুরুত্বপূর্ণ:** (১) password আছে? (২) username/email খালি? — তারপরই (৩) নতুন entity, (৪) field কপি + hash, (৫) `save` (INSERT; IDENTITY-র কারণে সাথে সাথে id পায়), (৬) DTO। চেক আগে, তাই ভুল request-এ কিছুই লেখা হয় না।

```
    public AdminResponse update(Long id, AdminRequest request) {
        Admin admin = getOrThrow(id);
        accounts.ensureUnique(request.username(), request.email(), id);

        apply(admin, request);
        // saveAndFlush runs the UPDATE now, so the response already shows the new updatedAt
        return AdminResponse.from(adminRepository.saveAndFlush(admin));
    }
```

লাইন 45–52 আগে খুঁজে বের করা (না পেলে 404), তারপর নিজেকে বাদ দিয়ে uniqueness, তারপর নতুন মান কপি। Entity এই transaction-এ লোড হওয়ায় **managed** — Hibernate নিজে পরিবর্তন ধরে commit-এ UPDATE চালাত (**dirty checking**)। `saveAndFlush` UPDATE *এখনই* চালায়, ফলে `@PreUpdate` চলে আর ফেরত JSON-এ নতুন `updatedAt` থাকে।

```
    public void delete(Long id) {
        Admin admin = getOrThrow(id);
        // Without at least one admin nobody could ever create admins again
        if (adminRepository.count() <= 1) {
            throw new ConflictException("Cannot delete the last admin account");
        }
        adminRepository.delete(admin);
    }
```

লাইন 54–61 **শেষ admin-এর নিয়ম:** admin মাত্র একজন হলে মুছতে দেওয়া হয় না → 409। কারণ শুধু admin-ই admin বানাতে পারে; শেষজন মুছে গেলে admin-এর কাজ আর কখনো করা যেত না।

```
    private void apply(Admin admin, AdminRequest request) {
        accounts.applyCommonFields(admin, request.name(), request.email(), request.phone(),
                request.username(), request.password());
    }
```

লাইন 63–66 সাধারণ field কপি shared service-কে দিয়ে করানো (admin-এর বাড়তি field নেই)।

```
    private Admin getOrThrow(Long id) {
        return adminRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Admin", id));
    }
}
```

লাইন 68–72 `findById` একটা `Optional` দেয়; `orElseThrow(() -> new ResourceNotFoundException(...))` হয় ভেতরের admin বের করে, নয়তো exception ছোড়ে → 404। Lambda থাকায় exception object শুধু দরকার হলেই তৈরি হয়।

src/main/java/com/university/ums/service/TeacherService.java (67 লাইন)

```
package com.university.ums.service;

import com.university.ums.dto.TeacherRequest;
import com.university.ums.dto.TeacherResponse;
import com.university.ums.exception.ResourceNotFoundException;
import com.university.ums.model.Teacher;
import com.university.ums.repository.TeacherRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
```

লাইন 1–11 import।

```
@Service
@Transactional
public class TeacherService {

    private final TeacherRepository teacherRepository;
    private final UserAccountService accounts;

    public TeacherService(TeacherRepository teacherRepository, UserAccountService accounts) {
        this.teacherRepository = teacherRepository;
        this.accounts = accounts;
    }

    @Transactional(readOnly = true)
    public List<TeacherResponse> findAll() {
        return teacherRepository.findAll().stream().map(TeacherResponse::from).toList();
    }

    @Transactional(readOnly = true)
    public TeacherResponse findById(Long id) {
        return TeacherResponse.from(getOrThrow(id));
    }
```

লাইন 13–33 AdminService-এর মতোই: class-level `@Transactional`, dependency, `findAll` ও `findById` (readOnly)।

```
    public TeacherResponse create(TeacherRequest request) {
        accounts.requirePassword(request.password());
        accounts.ensureUnique(request.username(), request.email(), null);

        Teacher teacher = new Teacher();
        apply(teacher, request);
        return TeacherResponse.from(teacherRepository.save(teacher));
    }
```

লাইন 35–42 password চেক → uniqueness → entity → `apply` → save → DTO।

```
    public TeacherResponse update(Long id, TeacherRequest request) {
        Teacher teacher = getOrThrow(id);
        accounts.ensureUnique(request.username(), request.email(), id);

        apply(teacher, request);
        return TeacherResponse.from(teacherRepository.saveAndFlush(teacher));
    }
```

লাইন 44–50 খোঁজা → নিজেকে বাদ দিয়ে uniqueness → `apply` → `saveAndFlush`।

```
    public void delete(Long id) {
        teacherRepository.delete(getOrThrow(id));
    }
```

লাইন 52–54 খুঁজে মুছে ফেলা। Teacher-এর "শেষজন" নিয়ম নেই।

```
    private void apply(Teacher teacher, TeacherRequest request) {
        accounts.applyCommonFields(teacher, request.name(), request.email(), request.phone(),
                request.username(), request.password());
        teacher.setDepartment(request.department().trim());
        teacher.setDesignation(request.designation().trim());
    }
```

লাইন 56–61 `apply` সাধারণ field-এর পরে Teacher-এর নিজস্ব `department` ও `designation` (trim করে) বসায়।

```
    private Teacher getOrThrow(Long id) {
        return teacherRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Teacher", id));
    }
}
```

লাইন 63–67 না পেলে 404 "Teacher with id ... not found"।

src/main/java/com/university/ums/service/StudentService.java (75 লাইন)

```
package com.university.ums.service;

import com.university.ums.dto.StudentRequest;
import com.university.ums.dto.StudentResponse;
import com.university.ums.exception.ConflictException;
import com.university.ums.exception.ResourceNotFoundException;
import com.university.ums.model.Student;
import com.university.ums.repository.StudentRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
```

লাইন 1–12 import (বাড়তি: `ConflictException`)।

```
@Service
@Transactional
public class StudentService {

    private final StudentRepository studentRepository;
    private final UserAccountService accounts;

    public StudentService(StudentRepository studentRepository, UserAccountService accounts) {
        this.studentRepository = studentRepository;
        this.accounts = accounts;
    }

    @Transactional(readOnly = true)
    public List<StudentResponse> findAll() {
        return studentRepository.findAll().stream().map(StudentResponse::from).toList();
    }

    @Transactional(readOnly = true)
    public StudentResponse findById(Long id) {
        return StudentResponse.from(getOrThrow(id));
    }
```

লাইন 14–34 TeacherService-এর মতোই গঠন।

```
36      public StudentResponse create(StudentRequest request) {
37          accounts.requirePassword(request.password());
38          accounts.ensureUnique(request.username(), request.email(), null);
39          if (studentRepository.existsByStudentId(request.studentId().trim())) {
40              throw new ConflictException("Student ID '" + request.studentId().trim() + "' is already registered");
41          }
42
43          Student student = new Student();
44          apply(student, request);
45          return StudentResponse.from(studentRepository.save(student));
46      }
```

লাইন 36–46 **একমাত্র পার্থক্য:** username/email-এর পরে রোল নম্বরও চেক — `existsByStudentId` সত্য হলে 409 "Student ID '...' is already registered"।

```
48      public StudentResponse update(Long id, StudentRequest request) {
49          Student student = getOrThrow(id);
50          accounts.ensureUnique(request.username(), request.email(), id);
51          if (studentRepository.existsByStudentIdAndIdNot(request.studentId().trim(), id)) {
52              throw new ConflictException("Student ID '" + request.studentId().trim() + "' is already registered");
53          }
54
55          apply(student, request);
56          return StudentResponse.from(studentRepository.saveAndFlush(student));
57      }
```

লাইন 48–57 Update-এ রোল নম্বর চেক নিজেকে বাদ দিয়ে (`existsByStudentIdAndIdNot`)।

```
59      public void delete(Long id) {
60          studentRepository.delete(getOrThrow(id));
61      }
```

লাইন 59–61 মুছে ফেলা।

```
63      private void apply(Student student, StudentRequest request) {
64          accounts.applyCommonFields(student, request.name(), request.email(), request.phone(),
65                  request.username(), request.password());
66          student.setStudentId(request.studentId().trim());
67          student.setDepartment(request.department().trim());
68          student.setBatch(request.batch());
69      }
70
71      private Student getOrThrow(Long id) {
72          return studentRepository.findById(id)
73                  .orElseThrow(() -> new ResourceNotFoundException("Student", id));
74      }
75  }
```

লাইন 63–75 Student-এর নিজস্ব field: `studentId`, `department`, `batch`। তারপর `getOrThrow` → 404।

১৫. controller/ — REST endpoint

কাজ	Method + URL	সফল হলে	Idempotent?
সব দেখা	`GET /api/teachers`	200 + JSON array	হ্যাঁ
একটি দেখা	`GET /api/teachers/{id}`	200 (না থাকলে 404)	হ্যাঁ
তৈরি	`POST /api/teachers`	**201** + `Location`	না (দুবার = দুজন)
আপডেট	`PUT /api/teachers/{id}`	200 + নতুন অবস্থা	হ্যাঁ
মোছা	`DELETE /api/teachers/{id}`	**204**, body নেই	হ্যাঁ

REST নিয়ম: URL-এ **noun** (resource), বহুবচন, ছোট হাতের; কাজটা বোঝায় HTTP method — `/getTeacher` বা `/deleteStudent` নেই। *Idempotent* = একই request বারবার পাঠালেও ফলাফল একই অবস্থা।

src/main/java/com/university/ums/controller/TeacherController.java (74 লাইন)

```java
package com.university.ums.controller;

import com.university.ums.dto.TeacherRequest;
import com.university.ums.dto.TeacherResponse;
import com.university.ums.service.TeacherService;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;
```

লাইন 1–22 import: DTO, service, Swagger-এর `@Tag`, `@Valid`, Spring Web annotation, `ServletUriComponentsBuilder`, `URI`, `List`।

```java
/**
 * CRUD endpoints for teachers. Who may call them is decided in SecurityConfig
 * (/api/teachers/** rule), not here.
 */
```

লাইন 24–27 মন্তব্য: কে এই endpoint ডাকতে পারবে তা এখানে নয়, `SecurityConfig`-এ ঠিক হয়।

```java
@Tag(name = "Teachers", description = "Manage teachers (ADMIN or TEACHER)")
@RestController
@RequestMapping("/api/teachers")
public class TeacherController {
```

লাইন 28–31 `@Tag`: শুধু ডকুমেন্টেশন — Swagger UI-তে "Teachers" শিরোনামের নিচে সাজায়। `@RestController` = `@Controller` + `@ResponseBody`: প্রতিটি method-এর return সরাসরি JSON হয়ে response body-তে যায় (HTML view নয়)। `@RequestMapping("/api/teachers")`: class-এর সব method-এর URL-এর শুরু।

```java
    private final TeacherService teacherService;

    public TeacherController(TeacherService teacherService) {
        this.teacherService = teacherService;
    }
```

লাইন 33–37 শুধু service-এর উপর নির্ভর (constructor injection)। Controller কখনো repository বা entity ছোঁয় না।

```java
    // GET /api/teachers -> 200 OK + list
    @GetMapping
    public List<TeacherResponse> getAll() {
        return teacherService.findAll();
    }
```

লাইন 39–43 `@GetMapping` (path ছাড়া) = `GET /api/teachers`। `List<TeacherResponse>` ফেরত দিলে Jackson JSON array বানায়; সাধারণ object ফেরত দিলে status ডিফল্ট 200।

```java
    // GET /api/teachers/{id} -> 200 OK, or 404 Not Found
    @GetMapping("/{id}")
    public TeacherResponse getById(@PathVariable Long id) {
        return teacherService.findById(id);
    }
```

লাইন 45–49 `@GetMapping("/{id}")`: URL-এর `{id}` অংশ। `@PathVariable Long id`: `/api/teachers/5` থেকে 5 নিয়ে `Long`-এ বদলায়; "abc" হলে 400। না থাকলে service 404 ছোড়ে।

```
    // POST /api/teachers -> 201 Created + Location header pointing at the new resource
    @PostMapping
    public ResponseEntity<TeacherResponse> create(@Valid @RequestBody TeacherRequest request) {
        TeacherResponse created = teacherService.create(request);
```

লাইন 51–54 `@RequestBody` : JSON body → `TeacherRequest` । `@Valid` : method-এর ভেতরে ঢোকার আগে validation (ব্যর্থ হলে 400)। `ResponseEntity` ফেরত দেওয়া হয় কারণ status ও header নিজে ঠিক করতে হবে।

```
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}")
                .buildAndExpand(created.id())
                .toUri();
        return ResponseEntity.created(location).body(created);
    }
```

লাইন 55–60 নতুন resource-এর URL বানানো: বর্তমান URL (`http://localhost:8080/api/teachers`) + `/{id}`, তারপর `buildAndExpand(created.id())` দিয়ে `{id}`-এর জায়গায় আসল id → `…/api/teachers/7` ।
`ResponseEntity.created(location)` = status **201** + header `Location` ; `.body(created)` = নতুন teacher-এর JSON। Client জানল নতুন resource কোথায় আছে।

```
    // PUT /api/teachers/{id} -> 200 OK + updated resource
    @PutMapping("/{id}")
    public TeacherResponse update(@PathVariable Long id, @Valid @RequestBody TeacherRequest request) {
        return teacherService.update(id, request);
    }
```

লাইন 62–66 `@PutMapping("/{id}")` : id path থেকে, নতুন ডাটা body থেকে (validated)। 200 + আপডেট-হওয়া teacher।

```
    // DELETE /api/teachers/{id} -> 204 No Content
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        teacherService.delete(id);
    }
}
```

লাইন 68–74 `void` + `@ResponseStatus(HttpStatus.NO_CONTENT)` → **204**, খালি body। Controller-এ কোনো try/catch, SQL বা security চেক নেই — error global handler-এ, অনুমতি SecurityConfig-এ। একে বলে "thin controller"।

AdminController ও **StudentController** হুবহু একই গঠন — শুধু URL (`/api/admins` , `/api/students`), DTO, service ও `@Tag` -এর নাম আলাদা। শিক্ষক ওই ফাইল খুললে বলবেন: "TeacherController-এর মতোই, শুধু resource আলাদা।"

src/main/java/com/university/ums/controller/AuthController.java (79 লাইন)

```
package com.university.ums.controller;

import com.university.ums.dto.CurrentUserResponse;
import com.university.ums.dto.LoginRequest;
import com.university.ums.repository.UserRepository;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

লাইন 1–20 import: Security-র `AuthenticationManager` , token, context ও context repository; servlet request/response।

```
/**
 * Login and "who am I". Logout is handled by Spring Security's LogoutFilter
 * at POST /api/auth/logout (see SecurityConfig), so there is no method for it here.
 */
```

লাইন 22–25 Login ও "আমি কে"। Logout এখানে নেই, কারণ Spring Security-র `LogoutFilter` নিজেই সেটা সামলায় (SecurityConfig)।

```
@Tag(name = "Authentication", description = "Log in, see who is logged in, log out")
@RestController
@RequestMapping("/api/auth")
public class AuthController {
```

লাইন 26–29 Swagger-এ "Authentication" দল; base URL `/api/auth`।

```
    private final AuthenticationManager authenticationManager;
    private final SecurityContextRepository securityContextRepository;
    private final UserRepository userRepository;

    public AuthController(AuthenticationManager authenticationManager,
                          SecurityContextRepository securityContextRepository,
                          UserRepository userRepository) {
        this.authenticationManager = authenticationManager;
        this.securityContextRepository = securityContextRepository;
        this.userRepository = userRepository;
    }
```

লাইন 31–41 তিনটি dependency: `AuthenticationManager` (credential যাচাই, SecurityConfig-এর bean), `SecurityContextRepository` (session-এ লগইন সেভ, একই bean যা filter chain ব্যবহার করে), `UserRepository` (user-এর বিস্তারিত)।

```
    // POST /api/auth/login {"username": "...", "password": "..."} -> 200 + JSESSIONID cookie, or 401
    @PostMapping("/login")
    public CurrentUserResponse login(@Valid @RequestBody LoginRequest request,
                                     HttpServletRequest httpRequest,
                                     HttpServletResponse httpResponse) {
```

লাইন 43–47 `POST /api/auth/login`। SecurityConfig-এ `permitAll()`, তাই লগইন ছাড়াই আসা যায় (না হলে লগইন করতে লগইন লাগত!)। `@Valid`: দুটো field ফাঁকা নয়। `HttpServletRequest/Response` লাগে session ও cookie-র জন্য।

```
        // 1. Check username + password (loads user via CustomUserDetailsService, compares BCrypt hash).
        //    Wrong credentials throw BadCredentialsException -> 401 in GlobalExceptionHandler.
        Authentication authentication = authenticationManager.authenticate(
                UsernamePasswordAuthenticationToken.unauthenticated(request.username(), request.password()));
```

লাইন 48–51 **ধাপ ১ — যাচাই:** `UsernamePasswordAuthenticationToken.unauthenticated(u, p)` একটা "এখনো যাচাই হয়নি" token। `authenticationManager.authenticate()` → `DaoAuthenticationProvider` → আমাদের `CustomUserDetailsService.loadUserByUsername` ডাটাবেস থেকে user ও hash আনে → `passwordEncoder.matches(টাইপ-করা, hash)` BCrypt দিয়ে মেলায়। মিললে ফেরত আসে **authenticated** token (ROLE_TEACHER ইত্যাদি সহ)। না মিললে বা user না থাকলে `BadCredentialsException` → GlobalExceptionHandler → 401। (Provider ইচ্ছা করে "user নেই"-কেও "bad credentials" বানায়, আর দুই ক্ষেত্রে সমান সময় নেয় — timing attack বন্ধ।)

```
        // 2. Session fixation protection: never keep a session id that existed before login.
        if (httpRequest.getSession(false) != null) {
            httpRequest.changeSessionId();
        }
```

লাইন 53–56 **ধাপ ২ — session fixation সুরক্ষা:** আগে থেকে session থাকলে (`getSession(false)` নতুন বানায় না, শুধু থাকলে দেয়) `changeSessionId()` দিয়ে নতুন id। না হলে আক্রমণকারী আগে থেকে জানা session id ভিকটিমকে ধরিয়ে দিয়ে লগইনের পর সেটা ব্যবহার করতে পারত।

```
58          // 3. Store the logged-in user in the HTTP session so later requests are authenticated
59          //    just by sending the JSESSIONID cookie.
60          SecurityContext context = SecurityContextHolder.createEmptyContext();
61          context.setAuthentication(authentication);
62          SecurityContextHolder.setContext(context);
63          securityContextRepository.saveContext(context, httpRequest, httpResponse);
64
65          return currentUser(authentication);
66      }
```

লাইন 58–66 **ধাপ ৩ — session-এ সেভ:** খালি `SecurityContext` বানিয়ে তাতে authentication রাখা, এই request-এর জন্য `SecurityContextHolder`-এ বসানো, আর `securityContextRepository.saveContext()` দিয়ে HTTP session-এ `SPRING_SECURITY_CONTEXT` নামে সেভ। **Spring Security 6 থেকে এই সেভ নিজে লিখতে হয়** — framework আর স্বয়ংক্রিয়ভাবে সেভ করে না। Tomcat response-এ পাঠায় `Set-Cookie: JSESSIONID=…; HttpOnly; SameSite=Lax`। শেষে 200 + user-এর তথ্য।

```
68      // GET /api/auth/me -> 200 with the current user's id, name, username and role
69      @GetMapping("/me")
70      public CurrentUserResponse me(Authentication authentication) {
71          return currentUser(authentication);
72      }
```

লাইন 68–72 `GET /api/auth/me`: method parameter-এ `Authentication` লিখলে Spring বর্তমান লগইন-করা user দিয়ে দেয়। নিয়ম `anyRequest().authenticated()`, তাই যেকোনো লগইন-করা user।

```
74      private CurrentUserResponse currentUser(Authentication authentication) {
75          return userRepository.findByUsername(authentication.getName())
76                  .map(CurrentUserResponse::from)
77                  .orElseThrow(() -> new IllegalStateException("Authenticated user no longer exists"));
78      }
79  }
```

লাইন 74–79 `authentication.getName()` = username → ডাটাবেস থেকে user → `CurrentUserResponse`। লগইন থাকা অবস্থায় user মুছে গেলে `IllegalStateException` (সীমাবদ্ধতা অধ্যায়ে আলোচিত)।

১৬. security/ — Spring Security

src/main/java/com/university/ums/security/CustomUserDetailsService.java (36 লাইন)

```
1  package com.university.ums.security;
2
3  import com.university.ums.model.AppUser;
4  import com.university.ums.repository.UserRepository;
5  import com.university.ums.service.UserAccountService;
6  import org.springframework.security.core.userdetails.User;
7  import org.springframework.security.core.userdetails.UserDetails;
8  import org.springframework.security.core.userdetails.UserDetailsService;
9  import org.springframework.security.core.userdetails.UsernameNotFoundException;
10 import org.springframework.stereotype.Service;
```

লাইন 1–10 import। খেয়াল: এখানে `User` মানে Spring Security-র `User` class (আমাদের entity `AppUser`)।

```
12 /**
13  * Tells Spring Security how to find a user by username during login.
14  * Spring then compares the typed password against the stored BCrypt hash itself.
15  */
```

লাইন 12–15 Spring Security শুধু এই একটা interface আমাদের দিয়ে বাস্তবায়ন করায়: "username দিলে তার hash ও authority দাও"। Password মেলানো Spring নিজে করে।

```java
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
```

লাইন 16–23 `@Service` bean। Spring Boot দেখে যে একটা `UserDetailsService` ও একটা `PasswordEncoder` bean আছে, তাই এগুলো দিয়ে নিজে `DaoAuthenticationProvider` বানায় (log-এ দেখা যায়: "Global AuthenticationManager configured with UserDetailsService bean...")।

```java
    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser user = userRepository.findByUsername(UserAccountService.normalize(username))
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));
```

লাইন 25–28 `loadUserByUsername`: username normalize করে খোঁজা (তাই লগইনে বড়/ছোট হাতের যেকোনোভাবে লেখা যায়); না পেলে `UsernameNotFoundException`।

```java
        // roles("TEACHER") becomes the authority "ROLE_TEACHER", which hasRole("TEACHER") checks
        return User.withUsername(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRole().name())
                .build();
    }
}
```

লাইন 30–36 Spring-এর builder দিয়ে `UserDetails`: username, stored hash, আর `roles("TEACHER")` — যা নিজে `ROLE_` যোগ করে authority বানায় `ROLE_TEACHER`, আর SecurityConfig-এর `hasRole("TEACHER")` ঠিক এটাই খোঁজে।

src/main/java/com/university/ums/security/JsonSecurityHandlers.java (62 লাইন)

```java
package com.university.ums.security;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.university.ums.dto.MessageResponse;
import com.university.ums.exception.ApiError;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.security.web.authentication.logout.LogoutSuccessHandler;
import org.springframework.stereotype.Component;

import java.io.IOException;
```

লাইন 1–18 import: Jackson `ObjectMapper`, Security-র তিনটি handler interface।

```java
/**
 * Security errors happen in filters, before any controller runs, so @RestControllerAdvice
 * cannot see them. These handlers write the JSON responses instead of Spring's default
 * HTML login page / browser pop-up.
 */
```

লাইন 20–24 **কেন GlobalExceptionHandler যথেষ্ট না?** 401/403 ঠিক হয় **filter**-এ, Spring MVC-র আগেই; `@RestControllerAdvice` শুধু controller-এর exception দেখে। তাই Security নিজের interface ডাকে: `AuthenticationEntryPoint` (401), `AccessDeniedHandler` (403), `LogoutSuccessHandler`। এগুলো না থাকলে Spring খালি body / ব্রাউজার pop-up / login পেজে redirect পাঠাত।

```java
@Component
public class JsonSecurityHandlers implements AuthenticationEntryPoint, AccessDeniedHandler, LogoutSuccessHandler {

    private final ObjectMapper objectMapper;

    public JsonSecurityHandlers(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }
```

লাইন 25–32 একটা class তিনটা interface-ই বাস্তবায়ন করে (`implements A, B, C`)। `@Component` → bean, SecurityConfig-এ inject হয়। Spring-এর shared `ObjectMapper` নেওয়া হয় যাতে তারিখের format সব জায়গায় একই থাকে।

```java
    // Not logged in (no session, no/invalid Basic credentials) -> 401 Unauthorized
    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
                         AuthenticationException authException) throws IOException {
        write(response, HttpStatus.UNAUTHORIZED, ApiError.of(401, "Unauthorized",
                "Authentication required. Log in at POST /api/auth/login", request.getRequestURI()));
    }
```

লাইন 34–40 `commence()` : লগইন নেই (session নেই, Basic ভুল/নেই) → 401 JSON, বার্তায় কোথায় লগইন করতে হবে।

```java
    // Logged in, but the role is not allowed for this URL -> 403 Forbidden
    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       AccessDeniedException accessDeniedException) throws IOException {
        write(response, HttpStatus.FORBIDDEN, ApiError.of(403, "Forbidden",
                "Your role is not allowed to access this resource", request.getRequestURI()));
    }
```

লাইন 42–48 `handle()` : লগইন আছে কিন্তু role মেলেনি → 403 JSON।

```java
    // POST /api/auth/logout finished (session invalidated, cookie deleted) -> 200 OK
    @Override
    public void onLogoutSuccess(HttpServletRequest request, HttpServletResponse response,
                                Authentication authentication) throws IOException {
        write(response, HttpStatus.OK, new MessageResponse("Logged out successfully"));
    }
```

লাইন 50–55 `onLogoutSuccess()` : LogoutFilter session বাতিল ও cookie মোছার পরে ডাকে → 200 `{"message":"Logged out successfully"}`।

```java
    private void write(HttpServletResponse response, HttpStatus status, Object body) throws IOException {
        response.setStatus(status.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        objectMapper.writeValue(response.getOutputStream(), body);
    }
}
```

লাইন 57–62 সাহায্যকারী: status বসানো, `Content-Type: application/json`, তারপর `objectMapper.writeValue` দিয়ে object-কে JSON বানিয়ে সরাসরি response stream-এ লেখা।

src/main/java/com/university/ums/security/SecurityConfig.java (82 লাইন)

```java
package com.university.ums.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.security.web.servlet.util.matcher.PathPatternRequestMatcher;
```

লাইন 1–16 import: HttpSecurity builder, BCrypt, filter chain, session context repository, `PathPatternRequestMatcher`।

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

লাইন 18–20 `@Configuration`: এই class bean বানায়। `@EnableWebSecurity`: আমাদের web security config চালু।

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http,
                                                   JsonSecurityHandlers jsonHandlers,
                                                   SecurityContextRepository securityContextRepository) throws
Exception {
        http
```

লাইন 22–26 `@Bean` method: `HttpSecurity` একটা builder, প্রতিটি call chain-এর একটা অংশ সেট করে; শেষে `http.build()` আসল `SecurityFilterChain` বানায়, যা Spring Boot প্রতিটি request-এ চালায়। Parameter-এ Spring আমাদের `JsonSecurityHandlers` ও `SecurityContextRepository` bean দেয়।

```
                // REST API used by Postman/curl/JS, not HTML forms. The session cookie is SameSite=Lax
                // (application.yml), so browsers do not send it on cross-site POST/PUT/DELETE.
                .csrf(AbstractHttpConfigurer::disable)
```

লাইন 27–29 **CSRF বন্ধ** (`AbstractHttpConfigurer::disable` = method reference)। কারণ ও নিরাপত্তা: অধ্যায় ৭ — session cookie `SameSite=Lax`, কোনো GET ডাটা বদলায় না।

```
                // RBAC: rules are checked top to bottom, first match wins
                .authorizeHttpRequests(auth -> auth
```

লাইন 31–32 **RBAC নিয়ম শুরু।** উপর থেকে নিচে মেলানো হয়, **প্রথম যেটা মেলে সেটাই খাটে**।

```
                        .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
```

লাইন 33 Login: শুধু POST, সবার জন্য খোলা (`permitAll`)।

```
                        .requestMatchers("/error").permitAll()
```

লাইন 34 `/error`: Spring Boot-এর ভেতরের error পেজ; খোলা না রাখলে অপ্রত্যাশিত error বিভ্রান্তিকর 401 হয়ে যেত।

```
                        // Swagger UI page and the OpenAPI description it reads (docs only, no data)
                        .requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
```

লাইন 35–36 Swagger পেজ ও তার OpenAPI JSON খোলা — লগইনের আগেই পেজ লোড হতে হবে (পেজ থেকেই লগইন করা হয়)। এগুলোতে শুধু ডকুমেন্টেশন, কোনো ডাটা নেই।

```
                        .requestMatchers("/api/admins/**").hasRole("ADMIN")
                        .requestMatchers("/api/teachers/**").hasAnyRole("ADMIN", "TEACHER")
                        .requestMatchers("/api/students/**").hasAnyRole("ADMIN", "STUDENT")
```

লাইন 37–39 **মূল role নিয়ম:** `/**` মানে এই path ও তার নিচের সব। `hasRole("ADMIN")` খোঁজে authority `ROLE_ADMIN`। URL-এর নিয়ম সব method (GET/POST/PUT/DELETE)-এ খাটে = পুরো CRUD = "manage"। Admin → সব; Teacher → শুধু teachers; Student → শুধু students।

```
                        .anyRequest().authenticated())
```

লাইন 40 বাকি সব (যেমন `/api/auth/me`) শুধু লগইন চায়। `anyRequest()` অবশ্যই শেষে — পরে কোনো নিয়ম দিলে Spring চালুই হয় না।

```
                // Logged-in user is kept in the HTTP session (JSESSIONID cookie)
                .securityContext(context -> context.securityContextRepository(securityContextRepository))
```

লাইন 42–43 লগইন-করা user HTTP session-এ রাখা ও লোড করা হবে (`HttpSessionSecurityContextRepository`)।

```
                // Also accept "Authorization: Basic ..." on every request (handy for Postman/curl)
                .httpBasic(basic -> basic.authenticationEntryPoint(jsonHandlers))
```

লাইন 45–46 **HTTP Basic**-ও গ্রহণ: প্রতিটি request-এ `Authorization: Basic …` (Postman/curl/Swagger Authorize)। এর entry point আমাদের JSON handler; ডিফল্টে `WWW-Authenticate` header যেত আর ব্রাউজার pop-up দেখাত।

```
                // We use our own JSON login endpoint (AuthController), not the HTML login page
                .formLogin(AbstractHttpConfigurer::disable)
```

লাইন 48–49 HTML login পেজ (formLogin) বন্ধ, কারণ আমাদের নিজস্ব JSON login endpoint আছে।

```
51                .logout(logout -> logout
52                        .logoutRequestMatcher(PathPatternRequestMatcher.withDefaults()
53                                .matcher(HttpMethod.POST, "/api/auth/logout"))
54                        .invalidateHttpSession(true)
55                        .clearAuthentication(true)
56                        .deleteCookies("JSESSIONID")
57                        .logoutSuccessHandler(jsonHandlers))
```

লাইন 51–57 **Logout:** `LogoutFilter` শুধু `POST /api/auth/logout` ধরে (GET নয়, তাই কোনো লিংক/ছবি দিয়ে কাউকে logout করানো যায় না)। কাজ: session বাতিল (`invalidateHttpSession`), context মোছা, ব্রাউজারকে `JSESSIONID` cookie মুছতে বলা, শেষে আমাদের handler → 200 JSON। পুরনো cookie কেউ রেখে দিলেও কাজ করবে না, কারণ সার্ভারে সেই session আর নেই।

```
59                .exceptionHandling(ex -> ex
60                        .authenticationEntryPoint(jsonHandlers)   // 401
61                        .accessDeniedHandler(jsonHandlers));      // 403
```

লাইন 59–61 401 ও 403-এর জন্য আমাদের JSON handler।

```
63            return http.build();
64        }
```

লাইন 63–64 Builder থেকে filter chain তৈরি।

```
66        @Bean
67        public SecurityContextRepository securityContextRepository() {
68            return new HttpSessionSecurityContextRepository();
69        }
```

লাইন 66–69 `SecurityContextRepository` bean — filter chain ও AuthController **একই** storage ব্যবহার করে, তাই login-এ সেভ করা context পরের request-এ filter খুঁজে পায়।

```
71        // BCrypt: slow, salted one-way hash. Same password -> different hash every time.
72        @Bean
73        public PasswordEncoder passwordEncoder() {
74            return new BCryptPasswordEncoder();
75        }
```

লাইন 71–75 `PasswordEncoder` bean = `BCryptPasswordEncoder` (ডিফল্ট cost 10)। পুরো প্রজেক্টে hash ও match এটাই করে।

```
77        // Built by Spring Boot from CustomUserDetailsService + PasswordEncoder; used by AuthController
78        @Bean
79        public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
80            return configuration.getAuthenticationManager();
81        }
82    }
```

লাইন 77–82 `AuthenticationManager` bean: Spring Boot যে manager বানিয়েছে (CustomUserDetailsService + BCrypt দিয়ে) সেটা বের করে দেয়, যাতে AuthController-এ inject করা যায়।

১৭. config/ — প্রথম admin ও Swagger

src/main/java/com/university/ums/config/DataInitializer.java (110 লাইন)

```
1  package com.university.ums.config;
2
3  import com.university.ums.model.Admin;
4  import com.university.ums.model.Student;
5  import com.university.ums.model.Teacher;
6  import com.university.ums.repository.AdminRepository;
7  import com.university.ums.repository.StudentRepository;
8  import com.university.ums.repository.TeacherRepository;
9  import com.university.ums.service.UserAccountService;
10 import org.slf4j.Logger;
11 import org.slf4j.LoggerFactory;
12 import org.springframework.beans.factory.annotation.Value;
13 import org.springframework.boot.CommandLineRunner;
14 import org.springframework.stereotype.Component;
15 import org.springframework.transaction.annotation.Transactional;
```

লাইন 1–15 import: Logger, `@Value`, `CommandLineRunner`।

```
/**
 * Only admins can create admins, so an empty database would lock everyone out.
 * On startup, if there is no admin yet, create one from app.admin.* settings
 * (set APP_ADMIN_USERNAME / APP_ADMIN_PASSWORD / APP_ADMIN_EMAIL in Docker).
 * If app.demo-data is true, also add demo users for the lab demo.
 */
```

লাইন 17–22 **Bootstrap সমস্যা:** `/api/admins`-এ ADMIN লাগে, কিন্তু খালি ডাটাবেসে কোনো admin নেই — কেউ কখনো লগইন বা কাউকে তৈরি করতে পারত না। তাই API-র বাইরে থেকে প্রথম admin বানাতে হয়।

```
@Component
public class DataInitializer implements CommandLineRunner {

    private static final Logger log = LoggerFactory.getLogger(DataInitializer.class);

    private final AdminRepository adminRepository;
    private final TeacherRepository teacherRepository;
    private final StudentRepository studentRepository;
    private final UserAccountService accounts;
```

লাইন 23–31 `CommandLineRunner`: Spring Boot পুরো চালু হওয়ার পরে (ডাটাবেস প্রস্তুত, bean তৈরি) একবার `run()` ডাকে। Logger দিয়ে console-এ বার্তা।

```
    @Value("${app.admin.username}")
    private String username;

    @Value("${app.admin.password}")
    private String password;

    @Value("${app.admin.email}")
    private String email;
```

লাইন 33–40 `@Value("${app.admin.username}")`: `application.yml`-এর মান inject। Docker-এ environment variable `APP_ADMIN_USERNAME` থাকলে সেটাই জেতে (অধ্যায় ১৮)।

```
    @Value("${app.demo-data:false}")
    private boolean demoData;
```

লাইন 42–43 `@Value("${app.demo-data:false}")`: demo user বানাবে কিনা। `:false` মানে property না থাকলে ডিফল্ট false। application.yml-এ true, টেস্টে false (কিছু টেস্ট ঠিক একজন admin আশা করে)।

```
    public DataInitializer(AdminRepository adminRepository, TeacherRepository teacherRepository,
                           StudentRepository studentRepository, UserAccountService accounts) {
        this.adminRepository = adminRepository;
        this.teacherRepository = teacherRepository;
        this.studentRepository = studentRepository;
        this.accounts = accounts;
    }
```

লাইন 45–51 Constructor injection: তিনটি repository ও UserAccountService।

```
    @Override
    @Transactional
    public void run(String... args) {
        // Runs only on an empty database, so restarts never create duplicates
        if (adminRepository.count() > 0) {
            return;
        }
```

লাইন 53–59 `@Transactional`: সব INSERT এক transaction-এ। **Idempotent:** কোনো admin আগে থেকেই থাকলে কিছুই করে না — শুধু নতুন (খালি) ডাটাবেসে চলে, তাই restart-এ ডুপ্লিকেট হয় না বা password reset হয় না।

```
60          Admin admin = new Admin();
61          accounts.applyCommonFields(admin, "System Administrator", email, null, username, password);
62          adminRepository.save(admin);
63          log.info("Created initial admin account '{}'", admin.getUsername());
64
65          if (demoData) {
66              createDemoUsers();
67          }
68      }
```

লাইন 60–68 প্রথম admin: shared `applyCommonFields` দিয়ে field (password BCrypt-এ hash), save, log। তারপর `demoData` true হলে `createDemoUsers()`।

```
70      /**
71       * Inserted in this order, so on a fresh database the ids are sequential:
72       * admins 1–2, teachers 3–5, students 6–8 (all roles share the users table's id counter).
73       */
74      private void createDemoUsers() {
75          admin("Nusrat Jahan", "admin2@university.edu", "admin2", "admin123");
76
77          teacher("Dr. Abdul Karim", "karim.cse@university.edu", "teacher1", "Computer Science and Engineering",
"Professor");
78          teacher("Farhana Akter", "farhana.eee@university.edu", "teacher2", "Electrical and Electronic Engineering",
"Associate Professor");
79          teacher("Tanvir Hasan", "tanvir.me@university.edu", "teacher3", "Mechanical Engineering", "Lecturer");
80
81          student("Rahim Uddin", "rahim.2107001@university.edu", "student1", "2107001", "Computer Science and
Engineering", 2021);
82          student("Sadia Islam", "sadia.2107002@university.edu", "student2", "2107002", "Computer Science and
Engineering", 2021);
83          student("Arif Hossain", "arif.2203015@university.edu", "student3", "2203015", "Electrical and Electronic
Engineering", 2022);
84
85          log.info("Created demo users: admin2, teacher1–3 (password teacher123), student1–3 (password student123)");
86      }
```

লাইন 70–86 `createDemoUsers()`: নির্দিষ্ট ক্রমে ঢোকানো হয় — ১ admin, ৩ teacher, ৩ student। সব role একই `users` টেবিলের AUTO_INCREMENT ভাগ করে, তাই নতুন ডাটাবেসে id ক্রমিক: admin 1–2, teacher 3–5, student 6–8। শেষে log-এ পাসওয়ার্ডের তথ্য।

```
88      private void admin(String name, String email, String username, String rawPassword) {
89          Admin admin = new Admin();
90          accounts.applyCommonFields(admin, name, email, null, username, rawPassword);
91          adminRepository.save(admin);
92      }
93
94      private void teacher(String name, String email, String username, String department, String designation) {
95          Teacher teacher = new Teacher();
96          accounts.applyCommonFields(teacher, name, email, null, username, "teacher123");
97          teacher.setDepartment(department);
98          teacher.setDesignation(designation);
99          teacherRepository.save(teacher);
100     }
101
102     private void student(String name, String email, String username, String studentId, String department, int batch)
{
103         Student student = new Student();
104         accounts.applyCommonFields(student, name, email, null, username, "student123");
105         student.setStudentId(studentId);
106         student.setDepartment(department);
107         student.setBatch(batch);
108         studentRepository.save(student);
109     }
110 }
```

লাইন 88–110 তিনটি সাহায্যকারী method: প্রতিটি entity বানায়, `applyCommonFields` দিয়ে সাধারণ field ও hash-করা password (teacher-দের `teacher123`, student-দের `student123`), নিজস্ব field (department, designation / studentId, department, batch) বসায়, তারপর নিজের repository দিয়ে save। মনে রাখুন: MySQL কখনো id পুনরায় ব্যবহার করে না, তাই মুছে-বানালে id বাড়তেই থাকে; ১–৮ আবার পেতে `docker compose down -v`।

src/main/java/com/university/ums/config/OpenApiConfig.java (49 লাইন)

```java
package com.university.ums.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.Operation;
import io.swagger.v3.oas.models.PathItem;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.responses.ApiResponse;
import io.swagger.v3.oas.models.responses.ApiResponses;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.List;
```

লাইন 1–15 import: Swagger/OpenAPI model class।

```java
/**
 * Describes the API for Swagger UI (http://localhost:8080/swagger-ui.html).
 * springdoc scans the controllers for endpoints; this bean adds the title, the
 * "Authorize" button (HTTP Basic) and the logout endpoint, which is a Spring Security
 * filter rather than a controller method, so the scanner cannot find it.
 */
```

লাইন 17–22 springdoc নিজেই সব controller endpoint খুঁজে পায়; এই bean উপরে তিনটা জিনিস যোগ করে: শিরোনাম, "Authorize" বোতাম (HTTP Basic), আর logout endpoint — যা controller-এ নেই বলে scanner দেখতে পায় না।

```java
@Configuration
public class OpenApiConfig {

    private static final String BASIC_AUTH = "basicAuth";
```

লাইন 23–26 Constant `BASIC_AUTH` = security scheme-এর নাম, দুই জায়গায় একই নাম লাগে।

```java
    @Bean
    public OpenAPI universityOpenApi() {
        return new OpenAPI()
```

লাইন 28–30 `OpenAPI` bean, যেটা springdoc শুরুর বিন্দু হিসেবে নেয়।

```java
                .info(new Info()
                        .title("University Management System API")
                        .version("1.0.0")
                        .description("""
                                CSE 3230 Lab 4. Log in either way:
                                1. Click **Authorize** and enter a username/password (HTTP Basic), or
                                2. Call **POST /api/auth/login**; the browser keeps the session cookie.

                                Seeded admin: `admin` / `admin123`."""))
```

লাইন 31–39 পেজের শিরোনাম, version, বর্ণনা (Markdown: `**bold**`)। Java text block (`"""…"""`) দিয়ে বহু-লাইনের string।

```java
                .components(new Components().addSecuritySchemes(BASIC_AUTH,
                        new SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("basic")))
                .addSecurityItem(new SecurityRequirement().addList(BASIC_AUTH))
```

লাইন 40–42 Security scheme ঘোষণা: HTTP + "basic" — এটাই **Authorize** বোতাম দেখায়। `addSecurityItem` সব endpoint-এ প্রয়োগ (তালার চিহ্ন)।

```java
                .path("/api/auth/logout", new PathItem().post(new Operation()
                        .tags(List.of("Authentication"))
                        .summary("Log out: invalidates the session and deletes the JSESSIONID cookie")
                        .responses(new ApiResponses().addApiResponse("200",
                                new ApiResponse().description("Logged out successfully")))));
    }
}
```

লাইন 43–49 হাতে logout endpoint বর্ণনা: POST, "Authentication" দলে, 200-এর বর্ণনা। এতে Swagger-এ logout-এর জন্যও Execute বোতাম আসে।

১৮. application.yml — কনফিগারেশন

src/main/resources/application.yml (42 লাইন)

```yaml
spring:
  application:
    name: university-management-system
```

লাইন 1–3 অ্যাপের নাম (log-এ দেখা যায়)।

```yaml
  datasource:
    # Defaults are for running on your laptop against a local MySQL.
    # In Docker, compose.yaml overrides them with SPRING_DATASOURCE_URL etc.
    url: jdbc:mysql://localhost:3306/university_db
    username: ums_user
    password: ums_password
```

লাইন 4–9 MySQL-এর JDBC URL, user, password। ডিফল্ট `localhost` — Docker ছাড়া চালানোর জন্য। Docker-এ compose এগুলো env variable দিয়ে বদলায়।

```yaml
  jpa:
    hibernate:
      # Create/alter tables from the @Entity classes on startup (fine for a lab project)
      ddl-auto: update
    open-in-view: false
    show-sql: false
```

লাইন 10–15 `ddl-auto: update` : চালুর সময় Hibernate entity আর টেবিল মিলিয়ে না-থাকা টেবিল/কলাম বানায়, ডাটা মোছে না (ল্যাবের জন্য ঠিক; বাস্তবে Flyway)। `open-in-view: false` : JSON লেখা পর্যন্ত DB session খোলা রাখে না — সব DTO-তে রূপান্তর service-এর transaction-এর ভেতরেই হয়। `show-sql: false` : SQL log বন্ধ (true করলে প্রতিটি SQL দেখা যায় — ডেমোতে কাজে লাগতে পারে)।

```yaml
server:
  port: 8080
  servlet:
```

লাইন 17–19 Port 8080।

```yaml
    session:
      timeout: 30m
      cookie:
        http-only: true   # JavaScript cannot read the session cookie
        same-site: lax    # cookie not sent on cross-site POST/PUT/DELETE (CSRF protection)
```

লাইন 20–24 Session ৩০ মিনিট নিষ্ক্রিয় থাকলে শেষ। Cookie `http-only` (JS পড়তে পারে না) ও `same-site: lax` (CSRF সুরক্ষা)।

```yaml
# First admin, created on startup only if no admin exists (see DataInitializer).
# Override with environment variables APP_ADMIN_USERNAME, APP_ADMIN_PASSWORD, APP_ADMIN_EMAIL.
app:
  admin:
    username: admin
    password: admin123
    email: admin@university.edu
  # Also create demo users on a fresh database (admin2, teacher1-3, student1-3). Set APP_DEMO_DATA=false to skip.
  demo-data: true
```

লাইন 26–34 আমাদের নিজস্ব property `app.admin.*` (DataInitializer পড়ে)। `admin123` শুধু ডেমোর জন্য।

```yaml
# Swagger UI (http://localhost:8080/swagger-ui.html)
springdoc:
  swagger-ui:
    tags-sorter: alpha            # Admins, Authentication, Students, Teachers
    operations-sorter: method     # GET, POST, PUT, DELETE order inside each group
    try-it-out-enabled: true      # request forms are editable without an extra click
    persist-authorization: true   # keep the Authorize credentials after a page reload
```

লাইন 36–42 Swagger UI সেটিং: দল বর্ণানুক্রমে, method ক্রমে; form আগে থেকেই সম্পাদনযোগ্য; Authorize-এর credential reload-এর পরেও থাকে (ব্রাউজারের localStorage-এ — শেয়ার করা কম্পিউটারে বন্ধ রাখুন)।

Relaxed binding — Docker কীভাবে মান বদলায়: environment variable-এর অগ্রাধিকার application.yml-এর চেয়ে বেশি, আর Spring নাম মিলিয়ে নেয় (বড় হাতের, ডট → underscore): `SPRING_DATASOURCE_URL` → `spring.datasource.url`, `APP_ADMIN_PASSWORD` → `app.admin.password`। তাই একই jar ল্যাপটপে (localhost) ও Docker-এ (host `db`) কোনো code না বদলেই চলে। টেস্টের `src/test/resources/application.yml` datasource-কে H2 (`MODE=MySQL`) দিয়ে বদলায় ও `create-drop` ব্যবহার করে (প্রতিবার নতুন টেবিল)।

১৯. Dockerfile ও compose.yaml

Dockerfile (25 লাইন)

```
# ---------- Stage 1: build the jar with Maven + JDK ----------
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
```

লাইন 1–3 **ধাপ ১ (build):** `FROM maven:3.9-eclipse-temurin-21 AS build` — Maven + পূর্ণ JDK-সহ image, নাম "build"। `WORKDIR /app`: container-এর ভেতরে কাজের ফোল্ডার।

```
# Copy only pom.xml first and download dependencies. Docker caches this layer,
# so later builds skip the download unless pom.xml changes.
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
```

লাইন 5–8 **Layer cache কৌশল:** আগে শুধু `pom.xml` কপি করে dependency নামানো (`dependency:go-offline`)। Dependency কম বদলায়, code বেশি; শুধু Java code বদলালে Docker এই layer cache থেকে নেয়, শত শত MB আবার নামাতে হয় না।

```
# Now copy the source code and build the executable jar (tests run separately)
COPY src ./src
RUN mvn -B -q package -DskipTests
```

লাইন 10–12 তারপর source কপি করে jar বানানো। `-B` batch mode, `-q` শান্ত, `-DskipTests`: টেস্ট আলাদাভাবে `./mvnw test`-এ চলে, image build দ্রুত থাকে।

```
# ---------- Stage 2: small runtime image with only the JRE + jar ----------
FROM eclipse-temurin:21-jre
WORKDIR /app
```

লাইন 14–16 **ধাপ ২ (runtime):** নতুন করে শুধু JRE-সহ ছোট image। চূড়ান্ত image-এ Maven, JDK, source code বা build cache কিছুই নেই — ছোট, দ্রুত, আক্রমণের সুযোগ কম। একেই বলে **multi-stage build**।

```
# Do not run the application as root inside the container
RUN groupadd --system spring && useradd --system --gid spring spring
```

লাইন 18–19 ডিফল্টে container root হিসেবে চলে। আলাদা সাধারণ user `spring` বানানো হয়, যাতে অ্যাপ কখনো আক্রান্ত হলেও ক্ষতি সীমিত থাকে।

```
COPY --from=build /app/target/university-management-system-1.0.0.jar app.jar
```

লাইন 21 ধাপ ১ থেকে **শুধু jar** কপি, নাম `app.jar`।

```
USER spring
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

লাইন 23–25 `USER spring`: এখন থেকে non-root। `EXPOSE 8080`: কোন port-এ শোনে তার ডকুমেন্টেশন (নিজে port খোলে না; compose-এর `ports` খোলে)। `ENTRYPOINT ["java","-jar","/app/app.jar"]`: container চালু হলে এই কমান্ড। JSON রূপের কারণে Java-ই মূল process (PID 1), তাই `docker stop`-এর signal সরাসরি পায় এবং Spring সুন্দরভাবে বন্ধ হয়।

compose.yaml (43 লাইন)

```
# Start everything:  docker compose up --build
# Stop:              docker compose down        (add -v to also delete the database volume)
# Values like ${DB_PASSWORD:-ums_password} come from a .env file if present, else the default after ":-".
```

লাইন 1–3 ব্যবহারের নির্দেশনা (মন্তব্য)। `${VAR:-default}`: `.env` ফাইল বা shell থেকে মান, না থাকলে `:-`-এর পরের ডিফল্ট।

```yaml
services:
  db:
    image: mysql:8.4
```

লাইন 5–7 দুটি service (container): `db` ও `app`। `image: mysql:8.4` : Docker Hub-এর অফিসিয়াল MySQL image।

```yaml
    environment:
      MYSQL_DATABASE: ${DB_NAME:-university_db}
      MYSQL_USER: ${DB_USER:-ums_user}
      MYSQL_PASSWORD: ${DB_PASSWORD:-ums_password}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:-root_password}
```

লাইন 8–12 `MYSQL_*` : প্রথমবার চালুর সময় image নিজে ডাটাবেস, অ্যাপের user (password-সহ) ও root password বানায়। আসল password `.env` -এ রাখা যায় (git-এ যায় না)।

```yaml
    volumes:
      - db-data:/var/lib/mysql          # data survives container restarts
```

লাইন 13–14 Named volume `db-data` MySQL-এর ফাইল ফোল্ডারে mount — container মুছলেও ডাটা থাকে। `docker compose down` ডাটা রাখে, `down -v` মুছে দেয়।

```yaml
    healthcheck:
      # -h 127.0.0.1 forces TCP, which only answers once MySQL has finished its first-time setup
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u$$MYSQL_USER -p$$MYSQL_PASSWORD --silent"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
```

লাইন 15–21 **Health check:** প্রতি ৫ সেকেন্ডে `mysqladmin ping` ; উত্তর দিলে "healthy"। `$$` = compose যেন নিজে substitute না করে; container-এর shell `$MYSQL_USER` বসায়। `-h 127.0.0.1` TCP সংযোগ জোর করে: প্রথম চালুতে MySQL image networking ছাড়া একটা অস্থায়ী সার্ভার চালায়; socket-এ ping আগেভাগে সফল হয়ে যেত, TCP ping শুধু আসল সার্ভার চালু হলে সফল হয়। `retries 20` , `start_period 20s` : শুরুতে সময় দেওয়া।

```yaml
    restart: unless-stopped
    # No "ports:" on purpose: only the app container can reach MySQL (over the compose network).
```

লাইন 22–23 `restart: unless-stopped` : crash হলে আবার চালু (নিজে বন্ধ না করলে)। `db` -তে কোনো `ports:` নেই ইচ্ছা করে — MySQL শুধু compose network-এর ভেতরে, ল্যাপটপ বা ইন্টারনেট থেকে পৌঁছানো যায় না।

```yaml
  app:
    build: .
```

লাইন 25–26 `build: .` : এই ফোল্ডারের Dockerfile থেকে image বানাও।

```yaml
    depends_on:
      db:
        condition: service_healthy      # wait until MySQL accepts connections
```

লাইন 27–29 `condition: service_healthy` : db-র health check পাস না করা পর্যন্ত app শুরু হবে না। শুধু `depends_on` হলে container *চালু* হওয়া পর্যন্ত অপেক্ষা করত, কিন্তু MySQL তৈরি হতে আরও কয়েক সেকেন্ড লাগে → Spring সংযোগ করতে না পেরে crash করত।

```yaml
    environment:
      # "db" is the service name above; compose's internal DNS resolves it to the MySQL container
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/${DB_NAME:-university_db}
      SPRING_DATASOURCE_USERNAME: ${DB_USER:-ums_user}
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-ums_password}
```

লাইন 30–34 `SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/…` — `db` হলো service-এর নাম; compose-এর DNS একে db container-এর IP-তে বদলায়। Container-এর ভেতরে `localhost` মানে app container নিজেই, তাই localhost চলত না। username/password relaxed binding দিয়ে Spring-এ যায়।

```yaml
      APP_ADMIN_USERNAME: ${ADMIN_USERNAME:-admin}
      APP_ADMIN_PASSWORD: ${ADMIN_PASSWORD:-admin123}
      APP_ADMIN_EMAIL: ${ADMIN_EMAIL:-admin@university.edu}
```

লাইন 35–37 প্রথম admin-এর credential (DataInitializer)।

```
38     ports:
39       - "8080:8080"                    # host:container
```

লাইন 38–39 `"8080:8080"` = *host port : container port*। এর ফলে `http://localhost:8080`-এ অ্যাপ পাওয়া যায়।

```
40     restart: unless-stopped
```

লাইন 40 app-ও crash হলে আবার চালু।

```
42 volumes:
43   db-data:
```

লাইন 42–43 একদম নিচে named volume `db-data`-এর ঘোষণা।

.env.example

```
1 # Copy to .env and change the values. docker compose reads .env automatically.
2 DB_NAME=university_db
3 DB_USER=ums_user
4 DB_PASSWORD=change_me
5 DB_ROOT_PASSWORD=change_me_too
6 ADMIN_USERNAME=admin
7 ADMIN_PASSWORD=change_me_admin
8 ADMIN_EMAIL=admin@university.edu
```

`.env.example` কপি করে `.env` বানিয়ে আসল পাসওয়ার্ড দিন; compose নিজে `.env` পড়ে।

.dockerignore

```
1 target/
2 .git/
3 .idea/
4 .vscode/
5 *.iml
6 .env
7 docs/
```

Build-এর সময় এগুলো Docker-এ পাঠানো হয় না: পুরনো build output, IDE ফাইল, গোপন `.env`, docs।

২০. স্বয়ংক্রিয় টেস্ট — UniversityApiTests.java

src/test/java/com/university/ums/UniversityApiTests.java (256 লাইন)

```
1  package com.university.ums;
2
3  import com.university.ums.model.Teacher;
4  import com.university.ums.repository.TeacherRepository;
5  import org.junit.jupiter.api.Test;
6  import org.springframework.beans.factory.annotation.Autowired;
7  import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
8  import org.springframework.boot.test.context.SpringBootTest;
9  import org.springframework.http.MediaType;
10 import org.springframework.mock.web.MockHttpSession;
11 import org.springframework.test.web.servlet.MockMvc;
12 import org.springframework.test.web.servlet.MvcResult;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import static org.assertj.core.api.Assertions.assertThat;
16 import static org.hamcrest.Matchers.containsString;
17 import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.httpBasic;
18 import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
19 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
20 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
21 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
22 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
23 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
24 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
25 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

লাইন 1–25 import: JUnit, Spring test, MockMvc, AssertJ, Hamcrest; `static` import করায় `get(...)`, `status()` ইত্যাদি সরাসরি লেখা যায়।

```
27 /**
28  * End-to-end tests through the real security filter chain, controllers, services and an H2 database.
29  * Each test runs in a transaction that is rolled back, so tests do not affect each other.
30  */
```

লাইন 27–30 End-to-end টেস্ট: আসল security filter, controller, service ও H2 ডাটাবেস দিয়ে।

```java
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class UniversityApiTests {

    @Autowired
    MockMvc mvc;

    @Autowired
    TeacherRepository teacherRepository;
```

লাইন 31–40 `@SpringBootTest` : **পুরো** অ্যাপ চালু হয় (সব bean, security, JPA) — integration test। `@AutoConfigureMockMvc` : `MockMvc` দেয়, যা নেটওয়ার্ক port না খুলেই নকল HTTP request পুরো filter chain দিয়ে পাঠায়। `@Transactional` : প্রতিটি টেস্ট শেষে rollback — এক টেস্টের ডাটা অন্য টেস্টে যায় না; চালুর সময়ের seeded admin সবসময় থাকে। `@Autowired` field: টেস্ট class-এ field injection চলে।

```java
    private static final String TEACHER_JSON = """
            {"name":"Dr. Karim","email":"karim@uni.edu","username":"karim","password":"secret123",
             "department":"CSE","designation":"Professor"}""";

    private static final String STUDENT_JSON = """
            {"name":"Rahim","email":"rahim@uni.edu","username":"rahim","password":"secret123",
             "studentId":"2007001","department":"CSE","batch":2020}""";

    private static final String ADMIN_JSON = """
            {"name":"Second Admin","email":"admin2@uni.edu","username":"admin2","password":"secret123"}""";
```

লাইন 42–51 টেস্টের JSON body, Java text block (`"""…"""`) দিয়ে। পরে `.replace(...)` দিয়ে ভিন্ন সংস্করণ বানানো হয়।

```java
    private MockHttpSession login(String username, String password) throws Exception {
        MvcResult result = mvc.perform(post("/api/auth/login")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"username\":\"" + username + "\",\"password\":\"" + password + "\"}"))
                .andExpect(status().isOk())
                .andReturn();
        return (MockHttpSession) result.getRequest().getSession(false);
    }
```

লাইন 53–60 সাহায্যকারী: আসল `POST /api/auth/login` পাঠায়, 200 আশা করে, আর তৈরি হওয়া session ফেরত দেয় — পরের request-এ `.session(session)` দিয়ে ব্যবহার করলে "লগইন-করা" অবস্থা।

```java
    // ---------- Authentication ----------

    @Test
    void requestWithoutLoginIsRejectedWith401Json() throws Exception {
        mvc.perform(get("/api/students"))
                .andExpect(status().isUnauthorized())
                .andExpect(jsonPath("$.status").value(401));
    }
```

লাইন 62–69 লগইন ছাড়া `/api/students` → 401, আর JSON-এ `status` 401 (`jsonPath("$.status")`)।

```java
    @Test
    void loginWithSeededAdminCreatesSessionAndMeWorks() throws Exception {
        MockHttpSession session = login("admin", "admin123");
        mvc.perform(get("/api/auth/me").session(session))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.username").value("admin"))
                .andExpect(jsonPath("$.role").value("ADMIN"));
    }
```

লাইন 71–78 seeded admin দিয়ে লগইন → সেই session-এ `/api/auth/me` → username "admin", role "ADMIN"। প্রমাণ: DataInitializer কাজ করে, session login কাজ করে।

```java
    @Test
    void loginWithWrongPasswordReturns401() throws Exception {
        mvc.perform(post("/api/auth/login")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"username\":\"admin\",\"password\":\"wrong\"}"))
                .andExpect(status().isUnauthorized())
                .andExpect(jsonPath("$.message").value("Invalid username or password"));
    }
```

লাইন 80–87 ভুল password → 401 ও নির্দিষ্ট অস্পষ্ট বার্তা।

```java
    @Test
    void logoutInvalidatesSession() throws Exception {
        MockHttpSession session = login("admin", "admin123");
        mvc.perform(post("/api/auth/logout").session(session))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.message").value("Logged out successfully"));
        assertThat(session.isInvalid()).isTrue();
        mvc.perform(get("/api/auth/me").session(new MockHttpSession()))
                .andExpect(status().isUnauthorized());
    }
```

লাইন 89–98 logout → 200 ও বার্তা; `session.isInvalid()` সত্য; তারপর নতুন session-এ `/me` → 401।

```java
    @Test
    void httpBasicAuthenticationAlsoWorks() throws Exception {
        mvc.perform(get("/api/admins").with(httpBasic("admin", "admin123")))
                .andExpect(status().isOk());
    }
```

লাইন 100–104 `.with(httpBasic("admin","admin123"))` Basic header যোগ করে → `/api/admins` 200।

```java
    // ---------- Swagger UI ----------

    @Test
    void swaggerUiAndApiDocsArePublic() throws Exception {
        mvc.perform(get("/swagger-ui/index.html"))
                .andExpect(status().isOk());
        mvc.perform(get("/v3/api-docs"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.paths['/api/teachers/{id}']").exists())
                .andExpect(jsonPath("$.paths['/api/auth/logout'].post").exists())
                .andExpect(jsonPath("$.components.securitySchemes.basicAuth.scheme").value("basic"));
    }
```

লাইন 106–117 লগইন ছাড়াই Swagger পেজ ও `/v3/api-docs` 200; JSON-এ আমাদের endpoint, logout path ও basic security scheme আছে।

```java
    // ---------- Password hashing ----------

    @Test
    void passwordIsStoredAsBcryptHashAndNeverReturned() throws Exception {
        mvc.perform(post("/api/teachers").with(httpBasic("admin", "admin123"))
                        .contentType(MediaType.APPLICATION_JSON).content(TEACHER_JSON))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.password").doesNotExist());

        Teacher saved = teacherRepository.findAll().get(0);
        assertThat(saved.getPassword()).startsWith("$2a$").isNotEqualTo("secret123");

        // and the new teacher can log in with the original password
        login("karim", "secret123");
    }

    // ---------- CRUD + status codes ----------
```

লাইন 119–135 Teacher তৈরি → response-এ `password` নেই; ডাটাবেস থেকে পড়ে hash `$2a$` দিয়ে শুরু ও "secret123"-এর সমান নয়; নতুন teacher আসল password দিয়ে লগইন করতে পারে।

```
137     @Test
138     void fullCrudLifecycleForStudent() throws Exception {
139         MockHttpSession admin = login("admin", "admin123");
140 
141         MvcResult created = mvc.perform(post("/api/students").session(admin)
142                         .contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON))
143                 .andExpect(status().isCreated())
144                 .andExpect(header().string("Location", containsString("/api/students/")))
145                 .andExpect(jsonPath("$.role").value("STUDENT"))
146                 .andReturn();
147         String location = created.getResponse().getHeader("Location");
148 
149         mvc.perform(get(location).session(admin))
150                 .andExpect(status().isOk())
151                 .andExpect(jsonPath("$.studentId").value("2007001"));
152 
153         mvc.perform(put(location).session(admin).contentType(MediaType.APPLICATION_JSON)
154                         .content(STUDENT_JSON.replace("\"batch\":2020", "\"batch\":2021")))
155                 .andExpect(status().isOk())
156                 .andExpect(jsonPath("$.batch").value(2021));
157 
158         mvc.perform(delete(location).session(admin))
159                 .andExpect(status().isNoContent());
160 
161         mvc.perform(get(location).session(admin))
162                 .andExpect(status().isNotFound());
163     }
```

লাইন 137–163 পুরো CRUD: POST → 201 + Location; সেই Location-এ GET → 200; PUT (batch বদলে) → 200; DELETE → 204; আবার GET → 404।

```
165     @Test
166     void invalidBodyReturns400WithFieldErrors() throws Exception {
167         mvc.perform(post("/api/students").with(user("a").roles("ADMIN"))
168                         .contentType(MediaType.APPLICATION_JSON)
169                         .content("{\"name\":\"\",\"email\":\"not-an-email\",\"username\":\"x y\"}"))
170                 .andExpect(status().isBadRequest())
171                 .andExpect(jsonPath("$.fieldErrors.email").exists())
172                 .andExpect(jsonPath("$.fieldErrors.name").exists())
173                 .andExpect(jsonPath("$.fieldErrors.batch").exists());
174     }
```

লাইন 165–174 `.with(user("a").roles("ADMIN"))` : লগইন না করেই নকল authenticated user — যেসব টেস্টের বিষয় লগইন না, সেখানে দ্রুত উপায়। ভুল body → 400, `fieldErrors` -এ email, name, batch।

```
176     @Test
177     void createWithoutPasswordReturns400() throws Exception {
178         mvc.perform(post("/api/teachers").with(user("a").roles("ADMIN"))
179                         .contentType(MediaType.APPLICATION_JSON)
180                         .content(TEACHER_JSON.replace("\"password\":\"secret123\",", "")))
181                 .andExpect(status().isBadRequest());
182     }
```

লাইন 176–182 body থেকে password সরিয়ে create → 400।

```
184     @Test
185     void duplicateUsernameAcrossRolesReturns409() throws Exception {
186         String studentNamedAdmin = STUDENT_JSON.replace("\"username\":\"rahim\"", "\"username\":\"Admin\"");
187         mvc.perform(post("/api/students").with(user("a").roles("ADMIN"))
188                         .contentType(MediaType.APPLICATION_JSON).content(studentNamedAdmin))
189                 .andExpect(status().isConflict());
190     }
```

লাইন 184–190 Student-এর username "Admin" (বড় হাতের) → 409: প্রমাণ uniqueness সব role মিলিয়ে ও case-insensitive।

```
192     @Test
193     void unknownIdReturns404AndBadIdReturns400() throws Exception {
194         mvc.perform(get("/api/teachers/9999").with(user("a").roles("ADMIN")))
195                 .andExpect(status().isNotFound());
196         mvc.perform(get("/api/teachers/abc").with(user("a").roles("ADMIN")))
197                 .andExpect(status().isBadRequest());
198     }
```

লাইন 192–198 অজানা id 9999 → 404; "abc" → 400।

```
200     @Test
201     void lastAdminCannotBeDeleted() throws Exception {
202         MockHttpSession admin = login("admin", "admin123");
203         MvcResult me = mvc.perform(get("/api/auth/me").session(admin)).andReturn();
204         String id = com.jayway.jsonpath.JsonPath.read(me.getResponse().getContentAsString(), "$.id").toString();
205 
206         mvc.perform(delete("/api/admins/" + id).session(admin))
207                 .andExpect(status().isConflict());
208     }
```

লাইন 200–208 `/me` থেকে নিজের id বের করে (`JsonPath.read`) নিজেকে মোছার চেষ্টা → 409, কারণ সে-ই একমাত্র admin।

```
210     // ---------- Role-based access control ----------
211 
212     @Test
213     void adminCanManageAllThreeResources() throws Exception {
214         MockHttpSession admin = login("admin", "admin123");
215         mvc.perform(post("/api/admins").session(admin).contentType(MediaType.APPLICATION_JSON).content(ADMIN_JSON))
216                 .andExpect(status().isCreated());
217 
218 mvc.perform(post("/api/teachers").session(admin).contentType(MediaType.APPLICATION_JSON).content(TEACHER_JSON))
219                 .andExpect(status().isCreated());
220 
221 mvc.perform(post("/api/students").session(admin).contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON))
222                 .andExpect(status().isCreated());
223     }
```

লাইন 210–221 Admin তিন resource-এই তৈরি করতে পারে (তিনটাই 201)।

```
224     @Test
225     void teacherCanManageTeachersOnly() throws Exception {
226         mvc.perform(post("/api/teachers").with(httpBasic("admin", "admin123"))
227                 .contentType(MediaType.APPLICATION_JSON).content(TEACHER_JSON)).andExpect(status().isCreated());
228         MockHttpSession teacher = login("karim", "secret123");
229 
230         mvc.perform(get("/api/teachers").session(teacher)).andExpect(status().isOk());
231         mvc.perform(post("/api/teachers").session(teacher).contentType(MediaType.APPLICATION_JSON)
232                         .content(TEACHER_JSON.replace("karim", "salma")))
233                 .andExpect(status().isCreated());
234 
235         mvc.perform(get("/api/students").session(teacher)).andExpect(status().isForbidden());
236 
237 mvc.perform(post("/api/students").session(teacher).contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON))
238                 .andExpect(status().isForbidden());
239         mvc.perform(get("/api/admins").session(teacher))
240                 .andExpect(status().isForbidden())
241                 .andExpect(jsonPath("$.status").value(403));
242     }
```

লাইন 223–240 Admin একজন teacher বানায় → teacher লগইন → teachers দেখা ও তৈরি পারে (200, 201); students দেখা/তৈরি → 403; admins → 403।

```
243     @Test
244     void studentCanManageStudentsOnly() throws Exception {
245         mvc.perform(post("/api/students").with(httpBasic("admin", "admin123"))
246                 .contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON)).andExpect(status().isCreated());
247         MockHttpSession student = login("rahim", "secret123");
248 
249         mvc.perform(get("/api/students").session(student)).andExpect(status().isOk());
250         mvc.perform(post("/api/students").session(student).contentType(MediaType.APPLICATION_JSON)
251                         .content(STUDENT_JSON.replace("rahim", "karima").replace("2007001", "2007002")))
252                 .andExpect(status().isCreated());
253 
254         mvc.perform(get("/api/teachers").session(student)).andExpect(status().isForbidden());
255         mvc.perform(delete("/api/admins/1").session(student)).andExpect(status().isForbidden());
256     }
257 }
```

লাইন 242–256 Student students দেখতে/তৈরি করতে পারে; teachers → 403; admin মোছা → 403। এই তিন টেস্ট মিলে অ্যাসাইনমেন্টের পুরো RBAC টেবিল প্রমাণ করে।

২১. demo.sh — curl দিয়ে লাইভ ডেমো

demo.sh (58 লাইন)

```
#!/usr/bin/env bash
# Walks through login, CRUD, RBAC and logout against a running app.
# Usage: ./demo.sh            (app must be running: docker compose up --build)
#        BASE=http://host:8080 ./demo.sh
```

লাইন 1–4 `#!` (shebang): bash দিয়ে চালাও। নিচে ব্যবহারের নিয়ম।

```
set -u
BASE=${BASE:-http://localhost:8080}
JAR=$(mktemp -d)
trap 'rm -rf "$JAR"' EXIT
```

লাইন 5–8 `set -u`: অসংজ্ঞায়িত variable হলে থামো। `BASE` URL (বদলানো যায়)। `mktemp -d`: cookie রাখার অস্থায়ী ফোল্ডার; `trap … EXIT`: শেষে মুছে দাও।

```
step() { printf '\n\033[1;34m== %s\033[0m\n' "$1"; }
# call <cookie-jar-name> <METHOD> <path> [json-body]: prints status code and body
call() {
  local jar="$JAR/$1" method=$2 path=$3 body=${4:-}
  if [ -n "$body" ]; then
    curl -s -o "$JAR/out" -w '%{http_code}' -b "$jar" -c "$jar" -X "$method" "$BASE$path" \
         -H 'Content-Type: application/json' -d "$body"
  else
    curl -s -o "$JAR/out" -w '%{http_code}' -b "$jar" -c "$jar" -X "$method" "$BASE$path"
  fi
  printf '  %s %s\n' "$method" "$path"; cat "$JAR/out"; echo
}
```

লাইন 10–21 `step`: রঙিন শিরোনাম ছাপে। `call <jar> <METHOD> <path> [body]`: curl চালায়; `-b/-c jar` = cookie jar পড়া/লেখা (ব্রাউজারের মতো `JSESSIONID` মনে রাখে); প্রতিটি role-এর আলাদা jar, তাই তিনটা লগইন একে অপরকে মুছে দেয় না; `-w '%{http_code}'` status code ছাপে; body থাকলে JSON header-সহ পাঠায়।

```
step "1. Not logged in -> 401"
call anon GET /api/students

step "2. Admin logs in (seeded account) -> 200 + JSESSIONID cookie"
call admin POST /api/auth/login '{"username":"admin","password":"admin123"}'
```

লাইন 23–27 ধাপ ১–২: লগইন ছাড়া 401, তারপর admin লগইন 200।

```
step "3. Admin creates a teacher and a student -> 201 Created"
call admin POST /api/teachers '{"name":"Dr. Karim","email":"karim@uni.edu","username":"karim","password":"secret123","department":"CSE","designation":"Professor"}'
call admin POST /api/students '{"name":"Rahim","email":"rahim@uni.edu","username":"rahim","password":"secret123","studentId":"2007001","department":"CSE","batch":2020}'

step "4. Duplicate username -> 409, invalid body -> 400"
call admin POST /api/students '{"name":"X","email":"x@uni.edu","username":"rahim","password":"secret123","studentId":"2007999","department":"CSE","batch":2020}'
call admin POST /api/students '{"name":"","email":"bad","username":"a b"}'
```

লাইন 29–35 ধাপ ৩–৪: teacher ও student তৈরি (201), ডুপ্লিকেট (409), ভুল body (400)।

```
step "5. Teacher logs in: teachers OK, students/admins -> 403"
call teacher POST /api/auth/login '{"username":"karim","password":"secret123"}'
call teacher GET /api/teachers
call teacher GET /api/students
call teacher GET /api/admins

step "6. Student logs in: students OK, teachers -> 403"
call student POST /api/auth/login '{"username":"rahim","password":"secret123"}'
call student GET /api/students
call student GET /api/teachers
```

লাইন 37–46 ধাপ ৫–৬: teacher ও student লগইন করে নিজের resource 200, অন্যেরটা 403।

```
step "7. Admin updates and deletes the student -> 200, 204, then 404"
ID=$(curl -s -b "$JAR/admin" "$BASE/api/students" | sed -n 's/.*"id":\([0-9]*\).*"username":"rahim".*/\1/p')
call admin PUT "/api/students/$ID" '{"name":"Rahim
Uddin","email":"rahim@uni.edu","username":"rahim","studentId":"2007001","department":"CSE","batch":2021}'
call admin DELETE "/api/students/$ID"
call admin GET "/api/students/$ID"
TID=$(curl -s -b "$JAR/admin" "$BASE/api/teachers" | sed -n 's/.*"id":\([0-9]*\).*"username":"karim".*/\1/p')
call admin DELETE "/api/teachers/$TID"
```

লাইন 48–54 ধাপ ৭: list থেকে `sed` দিয়ে id বের করে PUT (200), DELETE (204), আবার GET (404); teacher-ও মুছে দেয় — তাই স্ক্রিপ্ট বারবার চালানো যায়।

```
step "8. Logout -> 200, then the same client is no longer logged in -> 401"
call admin POST /api/auth/logout
call admin GET /api/auth/me
```

লাইন 56–58 ধাপ ৮: logout 200, তারপর `/me` 401।

২২. চালানো ও ডেমো দেখানো

কমান্ড (প্রজেক্ট ফোল্ডার থেকে)	কী করে
`cd ~/claude/university-management-system`	প্রজেক্ট ফোল্ডারে যাওয়া (`compose.yaml` এখানেই)
`docker compose up --build`	image বানিয়ে db + app চালু (`-d` দিলে background-এ)। `Started UmsApplication` দেখা গেলে প্রস্তুত।
`docker compose ps` / `docker compose logs -f app`	container-এর অবস্থা / app-এর log
`./demo.sh`	দ্বিতীয় terminal-এ: পুরো ডেমো (প্রতিটি request-এর status ও JSON)
`docker compose exec db mysql -uums_user -pums_password university_db -e "SELECT id, username, password FROM users"`	শিক্ষককে ডাটাবেসের BCrypt hash দেখানো
`./mvnw test`	১৬টি টেস্ট চালানো (MySQL লাগে না)
`docker compose down` / `down -v`	বন্ধ (ডাটা থাকে) / বন্ধ ও ডাটাবেস মুছে নতুন শুরু

ডেমো অ্যাকাউন্ট (নতুন ডাটাবেসে স্বয়ংক্রিয়ভাবে তৈরি)

ID	Username	Password	Role	Name / details
1	admin	admin123	ADMIN	System Administrator
2	admin2	admin123	ADMIN	Nusrat Jahan
3	teacher1	teacher123	TEACHER	Dr. Abdul Karim — CSE, Professor
4	teacher2	teacher123	TEACHER	Farhana Akter — EEE, Associate Professor
5	teacher3	teacher123	TEACHER	Tanvir Hasan — ME, Lecturer
6	student1	student123	STUDENT	Rahim Uddin — 2107001, CSE, 2021
7	student2	student123	STUDENT	Sadia Islam — 2107002, CSE, 2021
8	student3	student123	STUDENT	Arif Hossain — 2203015, EEE, 2022

id ক্রমিক কারণ তিন role একই `users` টেবিল ভাগ করে। ডেমোর আগে ১-৮ নিশ্চিত করতে: `docker compose down -v` তারপর `docker compose up --build`। Demo user বাদ দিতে `APP_DEMO_DATA=false`।

Swagger UI দিয়ে ব্রাউজারে

1. `http://localhost:8080/swagger-ui.html` খুলুন। দল: Admins, Authentication, Students, Teachers। (সরাসরি `http://localhost:8080` খুললে 401 JSON আসবে — এটা স্বাভাবিক, কারণ এটা শুধু API, কোনো ওয়েবপেজ নেই।)
2. **লগইন উপায় ১ (session):** Authentication → `POST /api/auth/login` → Execute (body-তে আগে থেকেই admin/admin123; teacher/student হিসেবে লগইনে বদলান)। ব্রাউজার cookie রেখে দেয়।
3. **লগইন উপায় ২ (Basic):** উপরে ডানে **Authorize** → username/password → Authorize → Close।
4. যেকোনো endpoint খুলে JSON/id বদলে **Execute** — status, body, header (যেমন `Location`) দেখা যায়।
5. RBAC দেখাতে: teacher হিসেবে লগইন করে `GET /api/students` → 403।
6. Logout: `POST /api/auth/logout` → Execute; তারপর যেকোনো সুরক্ষিত endpoint → 401।

> **একসাথে একটাই লগইন উপায় ব্যবহার করুন।** Session cookie আর ভিন্ন user-এর Basic header দুটোই থাকলে `BasicAuthenticationFilter` Basic দিয়ে আবার authenticate করে, তাই Authorize-এর user জেতে। অন্য role টেস্টের আগে Authorize → Logout করুন।

Postman: `POST http://localhost:8080/api/auth/login`, Body → raw → JSON; Postman cookie মনে রাখে। অথবা প্রতিটি request-এ Authorization → Basic Auth। **শিক্ষককে দেখানোর ক্রম:** README → SecurityConfig (role নিয়ম) → TeacherController → AuthController → Dockerfile ও compose.yaml → লাইভ Swagger → MySQL-এ hash → `./mvnw test`।

২৩. সম্ভাব্য ভাইভা প্রশ্ন ও উত্তর

১. REST কী? আপনার API কি RESTful?

REST একটা স্থাপত্যরীতি: ডাটা **resource** হিসেবে URL দিয়ে চেনা যায়, স্ট্যান্ডার্ড HTTP method (GET/POST/PUT/DELETE) দিয়ে কাজ, JSON-এ আদান-প্রদান, ফলাফল status code দিয়ে। আমাদের API তাই করে। সৎ সতর্কতা: REST stateless সার্ভার পছন্দ করে, কিন্তু session login সার্ভারে অবস্থা রাখে; JWT হলে পুরো stateless হতো। আমাদের HTTP Basic অবশ্য stateless।

২. PUT, PATCH, POST-এর পার্থক্য?

POST নতুন resource বানায় (idempotent নয়)। PUT নির্দিষ্ট URL-এর resource পুরোটা প্রতিস্থাপন করে (idempotent)। PATCH শুধু পাঠানো field বদলায়। আমরা পুরো object-সহ PUT ব্যবহার করি; শুধু password ঐচ্ছিক।

৩. Create-এ 201 ও Location কেন? Delete-এ 204 কেন?

201 বলে নতুন কিছু তৈরি হয়েছে, `Location` বলে সেটা কোথায়। মোছার পর ফেরত দেওয়ার কিছু নেই — 204 মানে "সফল, body খালি"।

৪. 401 আর 403-এর পার্থক্য?

401 = authenticate হয়নি, সার্ভার জানে না তুমি কে। 403 = কে জানে, কিন্তু role-এর অনুমতি নেই। `ExceptionTranslationFilter` ঠিক করে: anonymous হলে entry point (401), আসল user হলে access denied handler (403)।

৫. @SpringBootApplication কী করে?

`@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`: bean দেওয়া যায়, লাইব্রেরি দেখে নিজে সেটআপ, package-এর সব annotation-যুক্ত class খুঁজে পাওয়া।

৬. Dependency injection কী? Constructor injection কেন?

Object নিজের dependency নিজে বানায় না, Spring container দেয়। Constructor injection-এ dependency স্পষ্ট ও `final`, object সবসময় পূর্ণ তৈরি, টেস্টে নকল দেওয়া সহজ।

৭. JPA, Hibernate, Spring Data JPA — পার্থক্য?

JPA = specification (annotation ও interface)। Hibernate = বাস্তবায়ন, SQL বানায়। Spring Data JPA = তার উপরে, interface ও method-এর নাম থেকে repository বানায়।

৮. কোনো code ছাড়া findByUsername কীভাবে কাজ করে?

Spring Data method-এর নাম (`find…By` + `Username`) বিশ্লেষণ করে `WHERE username = ?` বানায়, আর চালুর সময় interface বাস্তবায়নকারী proxy class তৈরি করে।

৯. ডাটাবেস ডিজাইন ব্যাখ্যা করুন। Inheritance কেন?

Abstract `AppUser`, JOINED inheritance: সাধারণ field `users` টেবিলে, আর `admins` / `teachers` / `students` একই id (FK) দিয়ে যুক্ত। একটা users টেবিল থাকায় username/email সব role মিলিয়ে unique, login এক জায়গায় খোঁজে; child কলাম NOT NULL রাখা যায়।

১০. DTO কী? Entity কেন সরাসরি ফেরত দেন না?

API-র JSON-এর আকার নির্ধারণকারী সাধারণ object। Entity ফেরত দিলে password hash ফাঁস হতো, client id/role বদলাতে পারত (mass assignment), আর API ডাটাবেসের সাথে বাঁধা পড়ত।

১১. Validation কীভাবে কাজ করে?

Request record-এ `@NotBlank`, `@Email` ইত্যাদি; controller parameter-এ `@Valid` থাকলে method চলার আগে যাচাই। ব্যর্থ হলে `MethodArgumentNotValidException` → GlobalExceptionHandler → 400 + `fieldErrors`।

১২. @Transactional কী করে? মাঝপথে exception হলে?

Method একটা transaction-এ চলে: সফল হলে commit, `RuntimeException`-এ rollback। যেমন users-এ INSERT-এর পর teachers-এ INSERT ব্যর্থ হলে দুটোই বাতিল — অর্ধেক teacher কখনো থাকে না।

১৩. Password কীভাবে রাখা হয়? আমাকে কারো password দেখাতে পারবেন?

BCrypt hash হিসেবে (`$2a$10$…`, ৬০ অক্ষর)। কেউই — admin বা developer-ও — আসল password বের করতে পারবে না; hash একমুখী। Login-এ টাইপ করা password একই salt দিয়ে hash করে তুলনা হয়।

১৪. Salt কী? MD5/SHA-256 নয় কেন?

Salt = প্রতিটি hash-এ মেশানো র‍্যান্ডম ডাটা, একই password-এর hash আলাদা হয়, rainbow table অকেজো। BCrypt ইচ্ছাকৃত ধীর (cost 10 = 1024 রাউন্ড) ও salt ভেতরে রাখে; MD5/SHA দ্রুত, তাই সেকেন্ডে কোটি অনুমান সম্ভব।

১৫. লগইন করলে ধাপে ধাপে কী হয়?

Login endpoint → AuthenticationManager → DaoAuthenticationProvider → CustomUserDetailsService user ও hash আনে → BCrypt `matches` → ROLE_X-সহ authenticated token → নতুন session id → SecurityContext session-এ সেভ → 200 + JSESSIONID cookie।

১৬. পরের request-এ সার্ভার কীভাবে জানে আমি কে?

Client `JSESSIONID` cookie পাঠায়; `SecurityContextHolderFilter` সেটা দিয়ে সার্ভারের session খুঁজে login-এ সেভ করা SecurityContext লোড করে।

১৭. Logout কীভাবে কাজ করে? Controller-এ তো logout method নেই।

SecurityConfig-এ Spring-এর `LogoutFilter` `POST /api/auth/logout`-এর জন্য সেট করা। সে session বাতিল করে, context মোছে, cookie মুছতে বলে, তারপর আমাদের handler JSON পাঠায়। সার্ভারে session না থাকায় পুরনো cookie আর কাজ করে না।

১৮. RBAC কোথায়? Teacher কেন student দেখতে পারে না দেখান।

`SecurityConfig`-এ: `/api/students/**` → `hasAnyRole("ADMIN","STUDENT")`। Teacher-এর শুধু ROLE_TEACHER, তাই AuthorizationFilter controller-এর আগেই থামায় → 403। প্রমাণ: টেস্ট `teacherCanManageTeachersOnly`।

১৯. hasRole আর hasAuthority-র পার্থক্য? নিয়মের ক্রম কি গুরুত্বপূর্ণ?

`hasRole("ADMIN")` নিজে `ROLE_` যোগ করে `ROLE_ADMIN` খোঁজে; `hasAuthority("ROLE_ADMIN")` একই জিনিস পুরো লেখা। ক্রম গুরুত্বপূর্ণ: প্রথম মিলটাই খাটে; `anyRequest()` অবশ্যই শেষে।

২০. CSRF বন্ধ করলেন কেন? অনিরাপদ নয়?

CSRF token HTML form-এর জন্য; এটা Postman/curl/JS-এর JSON API। সুরক্ষা রাখা হয়েছে `SameSite=Lax` cookie দিয়ে (ব্রাউজার অন্য সাইটের POST/PUT/DELETE-এ cookie পাঠায় না) এবং কোনো GET ডাটা বদলায় না। Basic client CSRF-এ আক্রান্ত হয় না।

২১. Session fixation কী, কীভাবে আটকান?

আক্রমণকারী ভিকটিমকে নিজের জানা session id ধরিয়ে দেয়, লগইনের পর সেটা ব্যবহার করে। Login-এ `changeSessionId()` দিয়ে নতুন id দেওয়া হয়।

২২. HTTP Basic কি নিরাপদ?

Credential শুধু Base64 এনকোড করা, এনক্রিপ্ট নয়, প্রতি request-এ যায় — তাই শুধু HTTPS-এ নিরাপদ। টেস্টের সুবিধার জন্য রাখা।

২৩. প্রথম admin কীভাবে তৈরি হয়? Restart দিলে?

`DataInitializer` (CommandLineRunner) চালুর সময় `app.admin.*` থেকে admin বানায় — শুধু যদি কোনো admin না থাকে। Restart-এ কিছু হয় না।

২৪. শেষ admin মোছা যায় না কেন?

শুধু admin-ই admin বানাতে পারে; শেষজন মুছলে admin ব্যবস্থাপনা চিরতরে অসম্ভব। তাই 409।

২৫. দুজন একই মুহূর্তে একই username নিলে?

দুটো Java চেকই পাস করতে পারে, কিন্তু ডাটাবেসের UNIQUE constraint দ্বিতীয় INSERT ফেরায় → `DataIntegrityViolationException` → 409।

২৬. Username ছোট হাতের কেন?

"Rahim" আর "rahim" যেন দুটো account না হয়, আর MySQL (case-insensitive) ও H2 (case-sensitive)-এ একই আচরণ। Login-ও normalize করে।

২৭. Docker কী? Image আর container?

Docker অ্যাপ ও তার সব প্রয়োজনীয় জিনিস একটা image-এ প্যাক করে; container হলো সেই image-এর চলমান রূপ, host থেকে বিচ্ছিন্ন। একই image → সব মেশিনে একই আচরণ।

২৮. Multi-stage Dockerfile কেন?

Build-এ Maven + JDK লাগে, চালাতে শুধু JRE + jar। ধাপ ২ শুধু jar কপি করে, তাই চূড়ান্ত image ছোট এবং build tool বা source নেই।

২৯. Docker-এর ভেতরে অ্যাপ ডাটাবেস খুঁজে পায় কীভাবে?

Compose প্রাইভেট network ও প্রতিটি service-এর নামে DNS দেয়। URL `jdbc:mysql://db:3306/…`-এ `db` = service নাম। Container-এর ভেতরে `localhost` মানে app নিজেই।

৩০. depends_on ও healthcheck কী করে?

শুধু depends_on container চালু হওয়া পর্যন্ত অপেক্ষা করে, কিন্তু MySQL প্রস্তুত হতে আরও সময় লাগে। `condition: service_healthy`-এ `mysqladmin ping` সফল না হওয়া পর্যন্ত app শুরু হয় না।

৩১. ডাটা কোথায় থাকে? docker compose down দিলে?

Named volume `db-data`-তে। `down` container মোছে, volume রাখে; `down -v` volume-ও মোছে। MySQL-এর port খোলা হয়নি, তাই বাইরে থেকে কেউ সংযোগ করতে পারে না।

৩২. Compose-এর env variable Spring-এ কীভাবে যায়?

Relaxed binding: `SPRING_DATASOURCE_URL` → `spring.datasource.url`; env variable application.yml-এর মানকে override করে।

৩৩. টেস্টে H2 কেন, চালাতে MySQL কেন?

টেস্ট দ্রুত ও কিছু ইনস্টল ছাড়াই চলা উচিত; H2 মেমরিতে MySQL mode-এ চলে। আসল অ্যাপ MySQL-এ। আরও কঠোর বিকল্প: Testcontainers দিয়ে আসল MySQL-এ টেস্ট।

৩৪. Session না JWT — কোনটা ভালো?

Session: সহজ, Spring-এ built-in, logout তাৎক্ষণিক; কিন্তু সার্ভারে অবস্থা থাকে, একাধিক সার্ভারে shared session লাগে। JWT: stateless, মোবাইল/microservice-এ ভালো; কিন্তু বাড়তি code, মেয়াদের আগে বাতিল করা কঠিন। এক সার্ভারের ল্যাব অ্যাপে session সহজ ও নিরাপদ।

৩৫. @RestControllerAdvice কেন 401/403 সামলায় না?

ওগুলো Spring MVC-র আগে security filter-এ ঘটে; advice শুধু controller-এর exception দেখে। তাই `AuthenticationEntryPoint` ও `AccessDeniedHandler`।

৩৬. Spring Security 6-এ saveContext কেন নিজে ডাকতে হয়?

Spring Security 6 থেকে `SecurityContextHolderFilter` শুধু লোড করে, request শেষে নিজে সেভ করে না; হাতে লগইন করালে `saveContext` ডাকতে হয়।

৩৭. save আর saveAndFlush-এর পার্থক্য?

`save` SQL commit পর্যন্ত দেরি করতে পারে; `saveAndFlush` সাথে সাথে পাঠায়, ফলে `@PreUpdate` চলে আর response-এ নতুন `updatedAt` থাকে।

৩৮. Swagger/OpenAPI কী? পাবলিক docs পেজ কি ঝুঁকি?

OpenAPI = REST API বর্ণনার স্ট্যান্ডার্ড JSON; Swagger UI তা দেখায় ও request পাঠাতে দেয়; springdoc আমাদের controller থেকে বানায়। শুধু ডকুমেন্টেশন পাবলিক — পেজ থেকে পাঠানো প্রতিটি request একই authentication ও RBAC পার হয়। Production-এ সাধারণত বন্ধ রাখা হয় (`springdoc.swagger-ui.enabled=false`)।

৩৯. Teacher যদি শুধু নিজের প্রোফাইল সম্পাদনা করতে পারার কথা হয়?

Method security: `@EnableMethodSecurity`, তারপর service-এ লগইন-করা username আর লক্ষ্য record মেলানো, বা `@PreAuthorize("hasRole('ADMIN') or …isOwner(#id, authentication.name)")`। অ্যাসাইনমেন্ট বলেছে "teacher manage teachers", তাই লেখা অনুযায়ী পুরো CRUD করা হয়েছে।

৪০. ddl-auto: update কী? Production-এ ব্যবহার করবেন?

Entity থেকে চালুর সময় টেবিল/কলাম তৈরি করে। Production-এ schema পরিবর্তন versioned ও reviewed হওয়া উচিত — Flyway/Liquibase migration, আর `ddl-auto: validate`।

২৪. সীমাবদ্ধতা ও উন্নতির সুযোগ

এগুলো নিজে থেকে উল্লেখ করলে বোঝা যায় আপনি ডিজাইনটা বুঝেছেন।

সীমাবদ্ধতা	উন্নতি
Teacher যেকোনো teacher-কে (student যেকোনো student-কে) সম্পাদনা/মুছতে পারে — অ্যাসাইনমেন্টের হুবহু কথা	Method security দিয়ে মালিকানা চেক (প্রশ্ন ৩৯)
User মুছে গেলে বা password বদলালে তার পুরনো session timeout/logout পর্যন্ত চলে	Spring Session + session registry দিয়ে সেই user-এর session বাতিল
`GET /api/students` সব row একসাথে দেয়	Pagination: `Pageable` → `?page=0&size=20&sort=name`
`ddl-auto: update` দিয়ে schema	Flyway migration
HTTPS নেই; login-এ brute-force সুরক্ষা নেই	TLS-সহ reverse proxy (Nginx/Caddy); rate limit / কয়েকবার ভুলে lock
CSRF বন্ধ (SameSite Lax দিয়ে প্রতিস্থাপিত)	ব্রাউজার front-end হলে `CookieCsrfTokenRepository`
compose-এ ডেমো credential ডিফল্ট	`.env`-এ বাধ্যতামূলক মান / Docker secrets
Swagger UI পাবলিক; app container-এর health endpoint নেই	Production profile-এ Swagger বন্ধ; Actuator `/actuator/health`
টেস্ট H2-তে, MySQL-এ নয়	Testcontainers